When Do Tree-Based Methods Win?

How Dataset Characteristics Shape Ensemble Forecasting Performance

Kayra Işık (VU Student Number: 2891441)

supervisor: prof. dr. Michael Massmann

co-reader: dr. Yicong Lin

28 October 2026

Abstract

Boosted trees won the M5 forecasting competition, but M5 is one large, feature-rich dataset, and its ranking need not hold elsewhere. This thesis asks which measurable properties of a dataset explain when tree-based ensembles beat simple and statistical benchmarks in daily retail sales forecasting.

A conditional-mean framework splits sales into a stable store level, predictable variation and residual noise. Ensembles should gain when the available features recover variation that a baseline misses. Experiments on 1115 Rossmann stores in 2013–2015 change the panel size, the training history, the store composition and the feature information. The main comparisons use fixed-origin 42-day holdouts; the experiments with updated lags evaluate one-step forecasts separately.

With only their own sales history, XGBoost and LightGBM do worse than a store-day-of-week baseline and per-store Auto-ARIMA. With calendar, promotion and store features they cut RMSE by roughly 39% against that weekly baseline, and updated sales lags bring the cut close to 50%. More stores do not reliably help, and the gain from a longer training history mostly stops after twelve months.

A covariate-matched comparison on 50 stores at two origins changes this ranking. Given the same covariates as the trees, ARIMAX is nearly tied with LightGBM at the first origin, and the trees win at the second; paired block-bootstrap intervals show the same contrast. With 3, 6 or 12 months of training history the pattern holds, and ARIMAX is never significantly better than LightGBM. Covariates close much of ARIMA's gap to the trees, and what remains depends on period and specification. Models should therefore be compared under the same information rather than ranked by family.

Key words: retail demand forecasting, gradient boosting, LightGBM, XGBoost, dataset characteristics, Rossmann store sales, M5 competition.

VU VRIJE UNIVERSITEIT AMSTERDAM

Contents

1 Introduction **3**

2 Relation to the Literature **4**
2.1 The M Competition Series and the Shift Toward Machine Learning . 4
2.2 The M5 Competition: Data, Design, and Findings 5
2.3 Tree-Based Methods in Retail Demand Forecasting 6
2.4 Global Forecasting Models and Dataset Characteristics 7

3 Model and Framework **8**
3.1 Forecasting Model: Decomposing Retail Sales 8
3.2 Baseline Forecast and Its Error 9
3.3 Tree-Based Forecast and the Performance Gap 9
3.4 The Estimators: From a Single Tree to Boosted Ensembles 10
3.5 Statistical Benchmark Models 21
3.6 Dataset Characteristics as Determinants of the Performance Gap . . . 23
3.7 Testable Predictions of the Framework 23

4 Empirical Application: Rossmann Store Sales **25**
4.1 Data 25
4.2 Research Design 33
4.3 Results 38

5 Conclusion **62**
5.1 Reconciling the Rejected and Unsupported Predictions 63
5.2 Practical Implications 64
5.3 Limitations and Future Work 65

Use of Generative AI **66**

References **67**

1 Introduction

The M5 Forecasting Competition showed that gradient-boosted trees, and especially LightGBM, can do very well on large retail forecasting problems (Makridakis et al., 2022a). Because so many leading entries used these methods, M5 has become a reference point for later retail forecasting work. But M5 is one particular setting: daily Walmart product sales in three US states, with a large cross-section, many zero-sales days at product level, a long sales history, detailed price and calendar variables, and a single 28-day test period. It shows how LightGBM performs there. It does not tell us whether the same ranking holds in retail data with a different scale, level of aggregation, history length or feature set. This thesis asks which part of that setting actually drives the tree advantage.

> **Research Question:** *Which measurable properties of retail demand data explain when tree-based ensemble methods outperform simple baselines in retail sales forecasting?*

The thesis answers this question with a theoretical framework and experiments on the Rossmann Store Sales dataset. The framework writes the gap between an ensemble and a simple baseline in terms of two things: how much predictable within-store variation there is, and how much of it the observed features recover. The Rossmann experiments then move these quantities by changing the number of stores, the length of the training window, the store-type mix and the information in the feature set. None of these levers is the final explanation. Each one is a way to change, or reveal, a deeper property of the data. What should carry over to other datasets is that deeper property: the predictable variation the baseline leaves behind, how strong the seasonal benchmark is, and how much recent and recurring demand information the model can actually see.

From raw records to a forecasting problem. The raw data is a long panel of daily rows, one row per store and day. Each row has a date, a store identifier, the day's total sales and a few flags and attributes, such as whether the store was open, whether a promotion ran that day, and the store's format and assortment level (Section 4 gives the full list). To answer the research question, this panel is turned into two objects for every store–day pair (s, t): a target $y_{s,t}$, the quantity every model tries to predict, and a feature vector $\mathbf{x}_{s,t}$, the information a model may use to predict it. The baselines and the tree ensembles differ only in how much of $\mathbf{x}_{s,t}$ they see and how flexibly they turn it into a prediction $\hat{y}_{s,t}$. The framework in Section 3 makes this precise by splitting $y_{s,t}$ into the part a model with a given $\mathbf{x}_{s,t}$ could recover and the part it never could. Seen this way, the research question is not about algorithm names. It is about what each estimator is shown and what it does with it. This is also why the estimators in Section 3.4 are introduced on one running example instead of as an abstract list.

Contributions

The thesis makes three contributions. First, it builds a framework for the performance gap between tree-based ensembles and a store-mean baseline, and derives testable predictions from two quantities: how much predictable variation each series has, and how well the observed features recover it. Second, it applies the framework to daily sales from 1115 Rossmann stores over roughly 2.5 years. It compares linear regression, a single CART tree, bagging, Random Forest, XGBoost and LightGBM against store-level Auto-ARIMA, ETS and Theta. Rossmann is a useful contrast to M5 because it has fewer series, a shorter history and store-level instead of product-level targets. Third, the experiments vary the number of stores, the training length, the store-type mix, the availability of lags and the test period, and then translate the resulting differences back into measurable data properties, so that the answer is not tied to conditions specific to Rossmann.

Structure

Section 2 reviews the forecasting literature and the role of the M competitions. Section 3 builds the modelling framework. It separates the forecasting model from the estimators that try to recover it, introduces those estimators step by step on a simulated panel, from a single regression tree to the boosted ensembles, and derives the testable predictions. Section 4 applies the framework to the Rossmann data: it describes the data, sets out the experiments, reports the results and re-expresses them as data properties. Section 5 concludes.

2 Relation to the Literature

2.1 The M Competition Series and the Shift Toward Machine Learning

The M competitions matter here because much of the evidence for tree-based forecasting comes from them. They evaluate every method on the same datasets, test periods and accuracy measures, so their results can be compared in a way that scattered studies cannot. LightGBM's strong result in M5 (Makridakis et al., 2022a) is the starting point of this thesis: does that advantage survive when the retail data look different?

The early competitions favoured simple methods. In M1, statistically sophisticated models could not consistently beat exponential smoothing or naïve forecasts (Makridakis et al., 2022a), and M3 singled out the simple Theta method as one of its strongest performers across 3003 series (Assimakopoulos and Nikolopoulos, 2000). M4 marked the turning point. Its winner combined exponential smoothing with a recurrent neural network, and several machine-learning methods did well

across 100000 series, yet pure machine learning still trailed statistical combinations on average (Makridakis et al., 2020). The balance only changed in M5, where most of the top 50 Accuracy and Uncertainty submissions used LightGBM, deep neural networks, or combinations of the two, and beat the statistical benchmarks at most levels of the hierarchy (Makridakis et al., 2022a).

2.1.1 Why Machine-Learning Performance Changed Across the Competitions

Algorithms alone do not explain this shift. The tasks themselves changed. The early competitions were largely univariate series from mixed domains and frequencies, where a method had to infer level, trend, and seasonality from each series' own short history. These conditions suit exponential smoothing, ARIMA, Theta, and forecast combinations. Machine learning also got a raw deal in how it was applied. Models often treated each series in isolation, which left few observations to estimate non-linear relationships and a high risk of fitting noise, and the strongest methods of the period pre-dated the regularisation, subsampling, and efficient training of modern XGBoost and LightGBM (Chen and Guestrin, 2016; Ke et al., 2017). M4 sat in between. Its 100000 series gave enough scale for global and hybrid learning, but its mix of domains and thin set of common predictors kept pure machine-learning results uneven (Makridakis et al., 2020).

M5 changed both the data and the modelling strategy. Its 30490 bottom-level series came from one retail system, linked through products, departments, stores, states, and calendar time, so participants could pool them into global models and feed in sales lags, rolling statistics, prices, events, and promotions. Boosted trees suddenly had a large tabular sample of related commercial behaviour, a setting where learning non-linear interactions pays off and short isolated series do not. The shift therefore reflects the data environment as much as the algorithms, and that is what motivates the research question, since changing the conditions may change the tree-ensemble advantage with them. M5 is itself only one setting, Walmart sales in the United States, with prices, promotions, holidays, and a product–store hierarchy full of strong weekly and intermittent patterns. Its result need not carry to datasets with fewer series, shorter histories, or sparser features. Neural networks also did well in M5 but fall outside this thesis, which sticks to XGBoost and LightGBM so that the data conditions can be studied closely instead of being buried under an architecture comparison.

2.2 The M5 Competition: Data, Design, and Findings

2.2.1 Data and Competition Design

The M5 dataset records daily unit sales for 3049 products in 10 Walmart stores across California, Texas, and Wisconsin. That gives 30490 bottom-level product–

store series, which aggregate through categories, departments, stores, and states into 42840 series in total. The data span 29 January 2011 to 19 June 2016 (1969 daily observations per series), and participants forecast the next 28 days (Makridakis et al., 2022c). They also received calendar and event indicators and product-level prices. Accuracy was measured by the Weighted Root Mean Squared Scaled Error (WRMSSE), which scales each series' errors by its historical variation and weights series by dollar sales. Table 8 in Section 4.1 sets the dataset's structural properties against those of the data used here.

2.2.2 The Winning Approach: Global Bottom-Up LightGBM

Many leading submissions used a global bottom-up approach built on LightGBM (Ma and Fildes, 2022). Instead of one model per series, a single model was trained on observations pooled across all 30490 bottom-level series, usually with sales lags, rolling averages, calendar variables, prices, and promotions, and forecasts were produced at the bottom level and aggregated upward. Pooling let the model learn effects shared across products and stores, such as weekly patterns and price responses. Nor was the approach limited to the winning entry: every top-50 Accuracy team beat the organisers' strongest benchmark by at least 16% (Makridakis et al., 2022b; Ma and Fildes, 2022), and LightGBM featured in most of the prize-winning Uncertainty submissions producing probabilistic forecasts (Makridakis et al., 2022d; Ord, 2022).

2.2.3 A Qualification: Robustness Across Test Windows

The official ranking used a single 28-day window, so it cannot say whether the ordering holds at other origins. Ma and Fildes (2022) re-evaluated leading LightGBM-based methods over five rolling 28-day windows. Rankings shifted noticeably even with the dataset and methods held fixed, which means part of the measured difference rode on the choice of test window. They recommend multiple rolling test periods when assessing competition results, advice this thesis follows in Stage 4.

This thesis treats the boosted trees that led M5 as estimators of the predictable structure in the forecasting model, not as standalone algorithms. Their mechanics, and the steps that lead from a single regression tree to XGBoost and LightGBM, are therefore explained inside the framework, in Section 3.4.

2.3 Tree-Based Methods in Retail Demand Forecasting

2.3.1 Why Ensemble Trees Are Useful for Tabular Retail Data

Retail demand reflects several forces (average store level, weekly and annual seasonality, promotions, competition, and local shocks) whose effects interact, so a promotion may matter differently by store type or weekday. Trees pick up such interactions without anyone specifying them in advance. A tree can split on store type, then on promotion status and day of week, so that a boosted model learns

that promotions behave differently on Mondays in one store format than in another, where a linear regression would need the interaction written out by hand. And because splits depend on the ordering of values, not their units, lags, binary indicators, calendar fields, and continuous measures such as competitor distance all enter the same model without standardisation.

Trained globally, a tree model can also borrow information across series (Ma and Fildes, 2022). A store with few promotion periods of its own leans on other stores' observations when estimating a promotion effect, which helps most when stores share demand patterns and tails off as their behaviour diverges.

The two implementations used throughout, XGBoost (Chen and Guestrin, 2016) and LightGBM (Ke et al., 2017), are the current standard for this kind of tabular forecasting. Their mechanics are described in Section 3.4.

2.4 Global Forecasting Models and Dataset Characteristics

Global versus local models. The M5 winners were *global* models, meaning one model trained on observations pooled across every series. Montero-Manso and Hyndman (2021) argue that this is more than a convenience. Given enough complexity, a global model can match or outperform local ones even when the series are unrelated, because forcing one function to fit many series at once acts as regularisation. The benefit from pooling grows with the capacity the global model is given, and capacity is exactly what the boosted trees in this thesis are tuned on. Januschowski et al. (2020) go further and argue that the global–local split classifies forecasting methods better than the older machine-learning–statistical split. This is the comparison made here: global boosted trees against local statistical models fitted per store.

Why trees keep winning on tabular data. Grinsztajn et al. (2022) benchmark tree ensembles against deep learning across a large collection of tabular datasets, and boosted trees remain ahead on medium-sized problems. They trace this to three features of tabular data: irregular target functions, uninformative features, and feature spaces that are not rotation-invariant. A typical store-level retail feature matrix, with its mix of identifiers, calendar fields, and a handful of continuous variables, fits that profile almost exactly. So it is not surprising that the strongest M5 entries were tree-based, not neural.

Dataset characteristics and method performance. A separate strand of work ties forecasting accuracy to measurable properties of the data. Kang et al. (2017) place series in an instance space spanned by features such as seasonal strength, trend, and entropy, and show that the best-performing method changes systematically across that space. Spiliotis et al. (2020) run the same analysis on the M competition data and find that those datasets occupy a narrow region of the feature space, which

cautions against reading too much into competition rankings. This thesis takes that caution seriously but turns the approach around. Instead of comparing methods across many different datasets, it changes the relevant characteristics experimentally inside one dataset.

The next section makes the mechanism precise. It models the gap between a tree-based ensemble and a simple store-level baseline as a function of measurable properties of the data.

3 Model and Framework

How accurate a forecast can be depends on three things: the information available at the forecast origin, the predictable variation the benchmark leaves behind, and the estimation error that comes from a finite sample. The framework below puts these together. It organises the mechanisms; the Rossmann results cannot be used to identify the separate hidden demand components it describes.

Table 1: Notation used throughout the thesis.

Symbol	Meaning
$s, t; N, T$	Store and target-date indices; panel dimensions
$y_{s,t}, \mu_s$	Daily sales and stable store-level mean
$\mathcal{F}_{s,t}, \mathbf{x}_{s,t}$	Reference origin-available information; selected predictors
$\delta_{s,t}, u_{s,t}$	Predictable departure from the mean; residual relative to $\mathcal{F}$
$\delta^*(\mathbf{x}), \xi_{s,t}$	Feature-based conditional mean of δ; its inaccessible part
$f, \hat{f}; \hat{\delta}$	Regression function, fitted predictor and fitted departure
$\rho^2_\infty, R_{\text{rec}}$	Feature ceiling; realised recovery after estimation error
$\mathcal{T}, \mathcal{S}, n$	Training/test store–day pairs; test size
$p, v, \tau; M, m$	Feature count/index and threshold; ensemble size/iteration
κ, σ^2	Correlation and variance across training draws at fixed inputs
$\mathcal{L}, \eta$	Loss and learning rate
$J, j, \mathcal{J}, I_j, w_j$	Leaf count/index/set, leaf observations and weight
$g, h; L$	First/second loss derivatives; lag operator

3.1 Forecasting Model: Decomposing Retail Sales

For a specified origin and horizon, let $\mathcal{F}_{s,t}$ contain only information available when the forecast is issued. Write

$$y_{s,t} = \mu_s + \delta_{s,t} + u_{s,t}, \qquad \delta_{s,t} = \mathbb{E}[y_{s,t} \mid \mathcal{F}_{s,t}] - \mu_s, \tag{1}$$

where $\mathbb{E}[u_{s,t} \mid \mathcal{F}_{s,t}] = 0$. In words, μ_s is the store's normal level, $\delta_{s,t}$ is the part of the day's deviation from that level that could be predicted with the information available

at the origin, and $u_{s,t}$ is everything else. The residual mixes real local effects and pure noise, and the thesis does not try to separate the two. A local event can be systematic without being predictable at the origin. What counts as unpredictable therefore depends on the information set: with more information, part of u moves into δ. Because δ is measured around each store's own level, μ_s can be read as the store's long-run mean. Treating that level as stable over an evaluation window is an approximation.

3.2 Baseline Forecast and Its Error

The store-mean benchmark uses the training history:

$$\hat{y}_{s,t}^{\text{naive}} = \bar{y}_s = |T_s|^{-1} \sum_{t \in T_s} y_{s,t}. \tag{2}$$

Its error is

$$e_{s,t}^{\text{naive}} = (\mu_s - \bar{y}_s) + \delta_{s,t} + u_{s,t} \approx \delta_{s,t} + u_{s,t}. \tag{3}$$

Dropping the level-estimation error $(\mu_s - \bar{y}_s)$ is least safe when the history is short or the store's level drifts. Store-DOW, which also averages by weekday, already removes the weekly part of δ, so it is a harder benchmark than Store-mean.

3.3 Tree-Based Forecast and the Performance Gap

For a fitted training sample, the feature-based prediction is

$$\hat{y}_{s,t}^{\text{tree}} = \hat{\mu}_s + \hat{\delta}(\mathbf{x}_{s,t}), \tag{4}$$

with error, when level-estimation error is small,

$$e_{s,t}^{\text{tree}} \approx \delta_{s,t} - \hat{\delta}(\mathbf{x}_{s,t}) + u_{s,t}. \tag{5}$$

The comparison rests on three assumptions: (A1) the level-estimation errors are negligible, (A2) the residual has conditional mean zero as in Equation (1), and (A3) both fitting and prediction use only information available at the origin. Rolling one-step forecasts use updated information, so they are a different task from fixed-origin 42-day forecasting. Under these assumptions,

$$\text{MSE}^{\text{naive}} \approx \text{Var}(\delta) + \mathbb{E}[u^2], \tag{6}$$

$$\text{MSE}^{\text{tree}} \approx \mathbb{E}[(\delta - \hat{\delta})^2] + \mathbb{E}[u^2]. \tag{7}$$

The mean squared error keeps the squared bias of the forecast; using the error variance instead would drop that term. In general the first term of Equation (6) is $\mathbb{E}[\delta^2] = \text{Var}(\delta) + (\mathbb{E}[\delta])^2$; writing $\text{Var}(\delta)$ assumes that δ has mean zero over the evaluation period, which need not hold for a seasonal holdout. Subtracting the two gives

$$G = \text{MSE}^{\text{naive}} - \text{MSE}^{\text{tree}} \approx \text{Var}(\delta) - \mathbb{E}[(\delta - \hat{\delta})^2]. \tag{8}$$

For $\mathrm{Var}(\delta) > 0$, define realised recovery by

$$R_{\mathrm{rec}} = 1 - \frac{\mathbb{E}[(\delta - \hat{\delta})^2]}{\mathrm{Var}(\delta)}, \qquad G \approx \mathrm{Var}(\delta) R_{\mathrm{rec}}. \tag{9}$$

R_{rec} can be negative: a tree that fits δ badly does worse than the store mean. It is also not a squared correlation, because a prediction can move together with demand and still be biased or badly scaled. Let $\delta^*(\mathbf{x}) = \mathbb{E}[\delta \mid \mathbf{x}]$ and $\xi = \delta - \delta^*(\mathbf{x})$. Under out-of-sample orthogonality of ξ and the fitted estimation error,

$$\mathbb{E}[(\delta - \hat{\delta})^2] = \mathrm{Var}(\delta)(1 - \rho_\infty^2) + \mathbb{E}[(\delta^* - \hat{\delta})^2], \tag{10}$$

where $\rho_\infty^2 = \mathrm{Var}(\delta^*)/\mathrm{Var}(\delta) \in [0, 1]$. Consequently,

$$G \approx \mathrm{Var}(\delta)\rho_\infty^2 - \mathbb{E}[(\delta^* - \hat{\delta})^2]. \tag{11}$$

Equation (11) separates two things. The ceiling ρ_∞^2 is about information: how much of δ the features could explain at best. The second term is about estimation: how close the fitted model gets to that best. The two can move in opposite directions. More observations can lower the estimation error, but they can also change the mix of stores or add older periods that behave differently. Richer features raise the ceiling, but in a finite sample they do not guarantee a gain. Against Store-DOW, the room to improve is the variation left after the weekly pattern is removed. The property regressions later in the thesis measure associations; they do not identify δ or test the decomposition causally.

3.4 The Estimators: From a Single Tree to Boosted Ensembles

The estimation error in Equation (11) is itself a bias–variance trade-off, $\mathbb{E}[(\delta^* - \hat{\delta})^2] = \mathrm{Bias}(\hat{\delta})^2 + \mathrm{Var}(\hat{\delta})$, and the estimators compared in this thesis differ in which of the two terms they attack. This subsection introduces them as a connected sequence, not a list: each method exists because the one before it leaves one part of the estimation error large, and the sequence moves step by step toward estimators that keep both terms small.

Throughout, the notation is that of the forecasting model. The target is always daily sales $y_{s,t}$ and the input is always the feature vector $\mathbf{x}_{s,t}$. Each estimator therefore produces a fit $\hat{f}(\mathbf{x}_{s,t})$ of the regression function $f(\mathbf{x}_{s,t}) = \mu_s + \mathbb{E}[\delta_{s,t} \mid \mathbf{x}_{s,t}]$, which equals $\mu_s + \delta_{s,t}$ in Equation (1) only when the features reveal $\delta_{s,t}$ fully, and the estimate $\hat{\delta}(\mathbf{x}_{s,t})$ in Equation (4) is this fit net of the store level. Training observations are the store–day pairs $(s, t) \in \mathcal{T}$.

A running simulation. To make each step concrete, every estimator in the sequence is applied to the same simulated panel, generated from Equation (1) with

every component known. The panel has $N = 100$ stores and $T = 90$ days (the last 14 days held out) and store levels $\mu_s \sim \mathcal{N}(6000, 2000^2)$. The parameters were chosen to represent a sales scale broadly similar to Rossmann, rather than to provide an exact empirical calibration. Its predictable component $\delta_{s,t}$ has standard deviation 600 and is built from a day-of-week pattern, a promotion whose lift strengthens toward the weekend, and a continuous store attribute with a non-linear effect. The residual sums two independent Gaussian draws of standard deviation 300, so its standard deviation is $300\sqrt{2}$. No serially correlated latent component is simulated. Four irrelevant noise features are appended to $\mathbf{x}_{s,t}$. The panel is deliberately small and its δ deliberately interactive, so that estimation error is visible instead of washed out by data volume. Because the conditional mean $f(\mathbf{x})$ is known, the estimation error of each method can be split into Monte Carlo estimates of squared bias and variance on a fixed test panel, averaging over 20 independent training draws. Table 2 and Figure 6 collect the results that the following paragraphs walk through.

Relation to Rossmann. The simulation keeps the main features of the Rossmann data: store differences, weekly effects and promotion interactions. It is still not a copy of Rossmann. It has 100 stores and 90 consecutive days instead of 1115 stores and 2.5 years, every store is observed every day, promotions are drawn independently and the noise is Gaussian. Its 14-day holdout is also shorter than the 42-day windows used later. The simulation is there to show the mechanisms, not to confirm the empirical ranking. Plain gradient boosting is explained below but is not fitted as a separate estimator in this six-step illustration.

Why these implementations. Each estimator below is fitted with one specific R package, chosen so that the sequence isolates the mechanism it is meant to illustrate rather than differences in implementation quality. CART uses `rpart` (Therneau and Atkinson, 2023), the standard R implementation of Breiman et al. (1984)'s original algorithm. Bagging and the Random Forest both use `ranger` (Wright and Ziegler, 2017), a single package whose only difference between the two is the `mtry` feature-subsampling argument (Section 3.4.3). Running both from the same codebase rules out a library-specific explanation for their gap. `ranger` was preferred here over the classic `randomForest` package because it parallelises across trees, which matters once $M = 200$ trees are grown on every training draw in the sequence and again across the Rossmann experiments of Section 4. XGBoost and LightGBM use their own official R packages, `xgboost` and `lightgbm`, maintained by the original authors and cited here through the same algorithm papers already used above, Chen and Guestrin (2016) and Ke et al. (2017), since neither package has a separate methods paper. These are the reference implementations, but comparing them still mixes differences in tuning with differences in the algorithms. The fitted LightGBM also uses standard gradient boosting (GBDT) without GOSS sampling, so none of its performance can be credited to GOSS.

3.4.1 Regression Trees

Breiman et al. (1984) introduced Classification and Regression Trees (CART). A regression tree splits the input space through a sequence of binary partitions. Every observation in a terminal region receives that region's mean response. At each internal node the algorithm searches every feature x_v, $v \in \{1, \ldots, p\}$, and every candidate threshold τ. It divides the data into $R_1 = \{\mathbf{x} : x_v \leq \tau\}$ and $R_2 = \{\mathbf{x} : x_v > \tau\}$, and it keeps the pair (v, τ) that minimises the **residual sum of squares (RSS)** across the two child nodes:

$$\mathrm{RSS}(v, \tau) = \sum_{(s,t)\in\mathcal{T}:\, \mathbf{x}_{s,t}\in R_1} (y_{s,t} - \bar{y}_{R_1})^2 + \sum_{(s,t)\in\mathcal{T}:\, \mathbf{x}_{s,t}\in R_2} (y_{s,t} - \bar{y}_{R_2})^2 \tag{12}$$

where $\bar{y}_{R_1}, \bar{y}_{R_2}$ are the mean responses in the child nodes. The search is exhaustive over features and thresholds, but it is greedy over depth, because it never asks what a split implies two levels below. Each child node then repeats the same search from scratch, so different branches can use different features. Splitting recurses until a stopping rule halts it, such as a maximum depth or a minimum node size. An observation reaches its leaf only after a chain of conditional splits. A tree therefore encodes interactions without anyone specifying them in advance.

Depth changes approximation and sampling error. A depth-three tree has at most eight leaves. That is far too few to separate 100 different store levels, let alone the interactions on top of them, so its squared bias is large. A deeper tree removes most of that bias and lowers the test error. Textbooks usually say that a deeper tree trades lower bias for higher variance, but this is not a rule that holds at every depth. On this panel the measured variance actually falls, from 187 to 142 (in 10^3 sales^2).

Even a shallow tree can change from one training draw to the next. When two splits score almost the same, a small change in the data flips which one is chosen. Deep trees add more variation on top of that, from small leaves and unstable splits further down. Bagging, the next step, asks whether averaging can remove this remaining error. Throughout the sequence the interpretation follows the measured bias and variance, not an assumed trade-off.

Result on the simulated panel (CART). The deep tree reaches RMSE = 588, with squared bias 21 and variance 142 (10^3 sales^2), against RMSE = 1820 for the depth-three tree, whose squared bias of 2921 dwarfs every other entry in Table 2. Depth removes most of the squared bias and here also lowers the variance. The deep tree still has much more variance than the ensembles that follow.

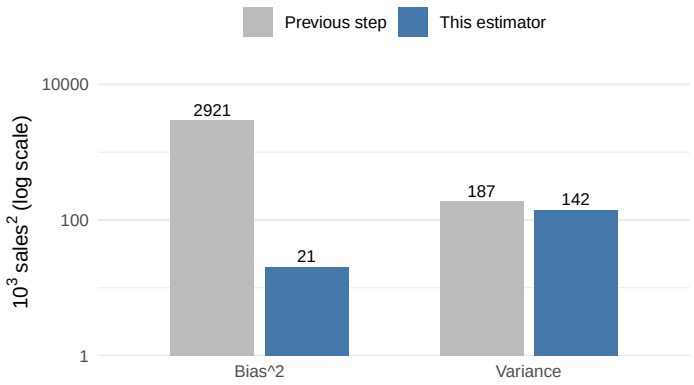


Figure 1: The deep tree against the depth-three tree. Depth sharply reduces squared bias; measured variance also falls on this panel.

3.4.2 Bagging

Breiman (1996) introduced Bootstrap Aggregating (bagging). The idea is simple. Instead of one deep tree, grow many deep trees and average their predictions,

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}^{*(m)}(\mathbf{x}), \tag{13}$$

where $\hat{f}^{*(m)}$ is the tree grown on the m-th sample. The average works because the trees share a target but not their errors. Every tree estimates the same regression function, so the signal survives the average. The noise each tree picks up from its own sample is its own, so it partly cancels.

The trees have to differ for this to help. CART is deterministic, so trees grown on the same data would come out identical and averaging them would change nothing. Each tree is therefore grown on a bootstrap sample, drawn from the training set with replacement. It sees roughly 63% of the distinct training observations, and its neighbours see a different subset.

The trees stay deep. Bagging is aimed at variance, although training on bootstrap samples and averaging can also change the bias a little (James et al., 2021).

On the simulated panel, bagging $M = 200$ deep trees cuts the deep tree's variance sharply, leaves the squared bias of the same small order, and improves the test RMSE (Table 2). The gain is still smaller than it could be. If the trees were independent, averaging would cut their common variance σ^2 to σ^2/M. They are not independent. The average pairwise correlation between their test predictions, reported as $\hat{\kappa}$ in Table 2, is close to one, because every tree recovers the store levels that drive most of the variation in this panel. This correlation is measured across test rows, so it mostly reflects the shared store levels. It is not the correlation of the trees' estimation errors that matters for the variance in the next subsection.

Result on the simulated panel (Bagging). Averaging $M = 200$ deep trees brings the RMSE down to 516. The variance falls from 142 (deep CART) to 43. The squared bias rises, from 21 to 41, but much less than the variance falls, so the total estimation error drops from 162 to 84. The correlation across test rows is about $\hat{\kappa} \approx 0.95$. It mostly reflects the store levels every tree picks up, not the error correlation across training draws in Equation (14).

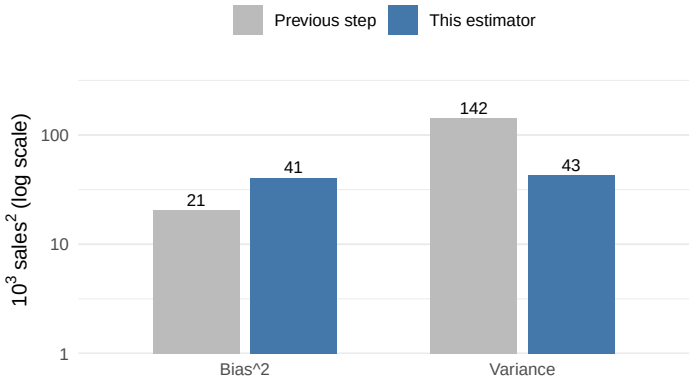


Figure 2: Bagging against the deep tree. Averaging cuts variance while increasing squared bias modestly in absolute terms.

3.4.3 Random Forests

That correlation is the next target. For M predictors with pairwise correlation κ and common variance σ^2, the variance of their average is

$$\operatorname{Var}\left(\frac{1}{M}\sum_{m=1}^{M}\hat{f}^{*(m)}(\mathbf{x})\right) = \kappa\,\sigma^2 + \frac{1-\kappa}{M}\,\sigma^2. \tag{14}$$

The second term vanishes as M grows. The first does not. Once the trees are strongly correlated, $\kappa\sigma^2$ is a floor that no number of extra trees can lower. (Here κ and σ^2 describe the predictions for the same input $\mathbf{x}$ across different training draws.)

Breiman (2001) addressed this with the Random Forest. It adds a second source of randomness on top of the bootstrap. At each node the algorithm draws a random subset of $p^* < p$ features and searches only those, a common regression heuristic being $p^* \approx p/3$ (James et al., 2021). The draw is repeated at every node, so a feature that is unavailable at one split can still be used at the next. The bootstrap subsamples the rows and this subsamples the columns. A strong predictor can no longer dominate every tree, and κ falls.

The simulation shows the intended effect and its cost together. The measured correlation $\hat{\kappa}$ falls to a fraction of its bagging value, and the variance edges below bagging's. The squared bias jumps far enough that the total estimation error ends up worse than bagging's (Table 2). The panel explains why. One feature, the

store identifier, carries most of the signal, and with $p^* \approx p/3$ it is absent from the candidate set at roughly two thirds of the splits. Those splits fall back on features that carry little. The trees are individually worse even though they are collectively more diverse. Decorrelation only pays when the hidden features have good substitutes. One could force the store identifier into every candidate set, but that was not tested here, and it would still not turn the forest into bagging, because the other features would still be subsampled. The same ordering, bagging ahead of the random forest, comes back in the empirical Stage 1 ranking in Section 4.

Result on the simulated panel (Random Forest). Feature subsampling lowers the tree correlation to $\hat{\kappa} \approx 0.70$, below bagging's 0.95, but total RMSE rises to 695, worse than bagging's 516: squared bias climbs to 257 while variance falls only to 33. Decorrelating the trees cost more in bias than it saved in variance on this panel, consistent with the cost of sometimes excluding the dominant store identifier.

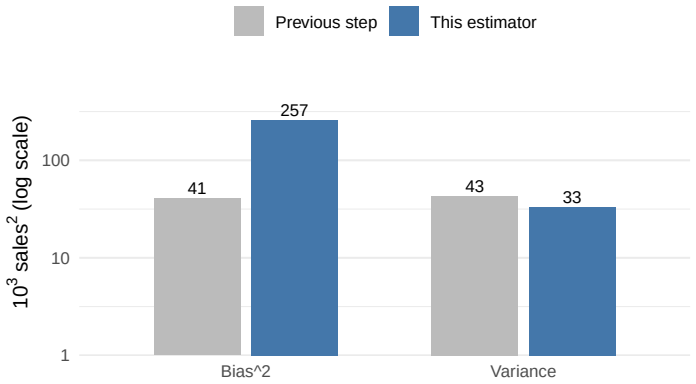


Figure 3: The random forest against bagging. Feature subsampling lowers the variance a little further and raises the squared bias by more.

3.4.4 Gradient Boosting

Bagging and the random forest both reach low bias the expensive way. They grow each tree deep and then pay for the variance that depth creates. Gradient boosting takes the opposite route (Friedman, 2001). Every tree is kept shallow, so its variance is small from the start, and the bias is removed sequentially instead of within one tree. Each new tree fits what the ensemble built so far still gets wrong.

The procedure is gradient descent in function space. For a differentiable loss $\mathcal{L}(y, \hat{f}(\mathbf{x}))$, the target of iteration m is the negative gradient of the loss with respect to the current prediction, known in the boosting literature as the pseudo-residual:

$$\tilde{r}_{s,t}^{(m)} = -\left[\frac{\partial \mathcal{L}\left(y_{s,t},\ \hat{f}^{(m-1)}(\mathbf{x}_{s,t})\right)}{\partial \hat{f}^{(m-1)}(\mathbf{x}_{s,t})}\right] \tag{15}$$

Here $\hat{f}^{(m-1)}(\mathbf{x}_{s,t})$ is the prediction of the whole ensemble after $m-1$ rounds, not of any single tree. The negative gradient points in the direction that prediction should move to lower the loss. Under squared-error loss it collapses to the ordinary residual $y_{s,t} - \hat{f}^{(m-1)}(\mathbf{x}_{s,t})$. Other losses, such as absolute error, Poisson deviance, or quantile loss, change the residuals but leave the procedure intact.

Each iteration fits a shallow tree f_m to these residuals. That tree is the descent step. It is then added to the ensemble, scaled by a **shrinkage parameter** (learning rate) $\eta \in (0, 1]$:

$$\hat{f}^{(m)}(\mathbf{x}) = \hat{f}^{(m-1)}(\mathbf{x}) + \eta\, f_m(\mathbf{x}) \tag{16}$$

A smaller η makes each tree contribute less, so more rounds are needed, and it usually generalises better. Every tree carries the noise of its own fit alongside the signal. Adding trees at full weight writes that noise into the model. Shrinkage keeps each update small. Together with the number of rounds and the tree size, it controls how much the model overfits. Unlike bagging, boosting does not rely on noise cancelling out across trees.

Where bagging averages away the noise of deep trees, boosting accumulates the signal of shallow ones: the depth-three tree that was hopeless on its own becomes the building block of the strongest estimators in the sequence.

3.4.5 XGBoost

Plain gradient boosting uses the slope of the loss, which tells the model which way to move. XGBoost also uses the curvature, which tells it how far (Chen and Guestrin, 2016). At each iteration it minimises a regularised second-order approximation of the loss:

$$\mathcal{L}^{(m)} \approx \sum_{(s,t)\in\mathcal{T}} \left[g_{s,t}\, f_m(\mathbf{x}_{s,t}) + \tfrac{1}{2}\, h_{s,t}\, f_m(\mathbf{x}_{s,t})^2 \right] + \Omega(f_m) \tag{17}$$

Here $g_{s,t}$ and $h_{s,t}$ are the first- and second-order derivatives of the loss at the current prediction, and $f_m(\mathbf{x}_{s,t})$ is the output of the new tree. The trees are still shallow and still grown level by level. What changes is how a split and a leaf value are scored.

The second change is the penalty $\Omega(f_m) = \gamma J + \frac{1}{2}\lambda \|w\|^2$, with J the number of leaves and w the vector of leaf weights. It pushes back against too many leaves and against large leaf values. The penalty sits inside the objective instead of being applied afterwards as pruning, so a split is taken only when its gain exceeds the cost it adds.

Together these give a closed-form optimal weight for each leaf j:

$$w_j^* = -\frac{\sum_{(s,t)\in I_j} g_{s,t}}{\sum_{(s,t)\in I_j} h_{s,t} + \lambda} \tag{18}$$

where I_j holds the training observations in leaf j. The numerator is the total correction the leaf is asked to make, and λ sits in the denominator. A leaf holding few observations has a small Hessian sum, so λ dominates and the leaf value is shrunk toward zero. Small and unstable leaves are quietened, and a single CART tree has no mechanism of this kind.

XGBoost also handles missing values during split selection, supports feature subsampling, and relies on data structures built for parallel computation. Together these made gradient boosting practical for large tabular datasets.

On the simulated panel the regularised boosting route pays off as the bias–variance logic predicts. Squared bias and variance are held small at the same time (Table 2).

Result on the simulated panel (XGBoost). RMSE = 468, squared bias 7, variance 30: the first estimator in the sequence to hold both terms down at once, undercutting every averaging-based method above (bagging's total estimation error is 84, XGBoost's is 37). Shallow, sequentially fitted trees with a regularised objective reach a bias–variance combination none of the deep-tree ensembles do.

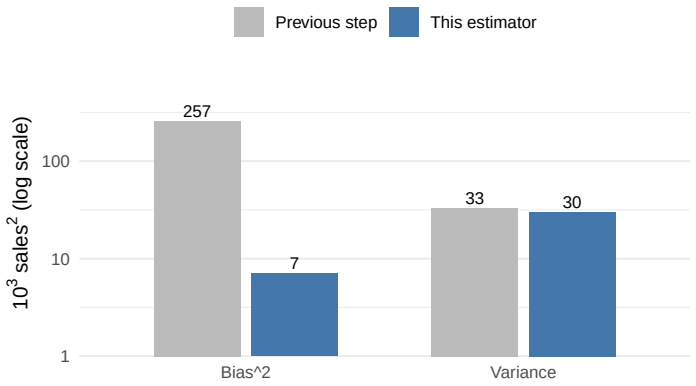


Figure 4: XGBoost against the random forest. Shallow trees fitted sequentially under a regularised objective hold both components down at once.

3.4.6 LightGBM

Ke et al. (2017) built LightGBM to make gradient boosting faster on large datasets. It changes three things: Gradient-based One-Side Sampling (GOSS), Exclusive Feature Bundling (EFB), and leaf-wise tree growth.

GOSS reduces the number of rows used to score a split. A large gradient means the model still fits that observation poorly, so those rows are the ones with something left to teach. GOSS keeps all of them and takes a sample of the rest. Writing $\mathcal{A}$ for the rows kept and $\mathcal{B}$ for the rows sampled, the gradient sum for region j is approximated as:

$$\tilde{G}_j = \sum_{(s,t)\in\mathcal{A}_j} g_{s,t} + \frac{1-a}{r} \sum_{(s,t)\in\mathcal{B}_j} g_{s,t} \tag{19}$$

where a is the retained fraction of all rows and r is the sampled low-gradient row count divided by the total row count. Using only part of the low-gradient rows would make the sum too small, so the factor $(1-a)/r$ scales that part back up. The sum then stays roughly unbiased, and splits can be scored on fewer rows.

EFB reduces the number of columns. Some features are almost never non-zero at the same time. Category indicators are the clearest case, because a product in one category is not in another. Two such features cannot be confused with each other, so LightGBM stores them in a single column. If the features are perfectly exclusive, nothing is lost; if some overlap is allowed, the bundle is an approximation. Either way there are fewer columns to scan.

Leaf-wise growth changes the tree shape. XGBoost, as configured here, grows trees depth-wise: it works through the tree level by level, although the stopping rules do not force every branch to the same depth. LightGBM instead always splits the leaf that promises the largest drop in loss. With $\mathcal{J}$ the set of leaves of the current tree, it selects:

$$j^* = \arg\max_{j\in\mathcal{J}} \Delta\mathcal{L}_j \tag{20}$$

where $\Delta\mathcal{L}_j$ is the loss reduction from splitting leaf j. The trees come out asymmetric. Some branches go deep and others stop early, so the tree spends its effort where the errors are largest. This overfits easily, because all the leaves can go into one region and fit its noise. The number of leaves, the minimum leaf size, the learning rate and the regularisation all still need watching.

On the simulated panel the two boosted implementations close the sequence with the smallest estimation errors (Table 2).

Result on the simulated panel (LightGBM). LightGBM ends with RMSE = 467, squared bias 10 and variance 23. Its total estimation error (33) is within a few points of XGBoost's (37) and the lowest, or tied for lowest, in the sequence. Starting from the deep tree, whose total error is 162, the sequence more than halves the estimation error by the time it reaches the two boosted models. Whether LightGBM's small edge over XGBoost is real or just replication noise can be read off Table 3 (row XGBoost vs LightGBM): $t = 1.23$, $p = 0.235$.

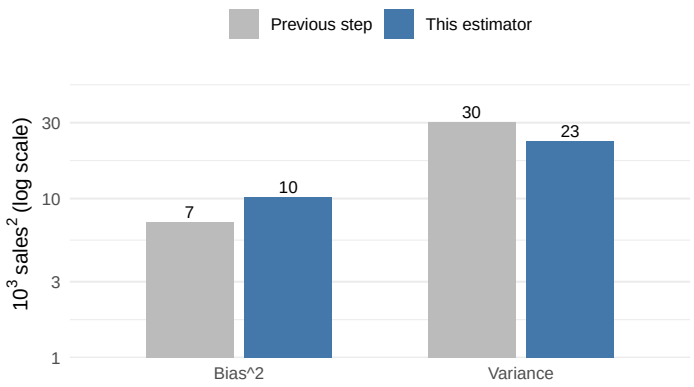


Figure 5: LightGBM against XGBoost. The two boosted implementations close the sequence at a similar place.

3.4.7 The Sequence at a Glance

Table 2 and Figure 6 report what each step of the sequence above does to the two components of estimation error on the simulated panel.

Table 2: The estimator sequence on the simulated panel of Section 3.4. Squared bias and variance of $\hat{f}(\mathbf{x})$ against the known conditional mean on a fixed test panel, averaged over 20 independent training draws (in 10^3 sales^2); RMSE is the square root of pooled mean test squared error; $\hat{\kappa}$ is the average pairwise correlation between individual trees' test predictions.

Estimator	RMSE	Bias^2	Variance	Bias^2+Var	$\hat{\kappa}$
CART (depth 3)	1820	2921	187	3108	–
CART (deep)	588	21	142	162	–
Bagging (M=200)	516	41	43	84	0.95
Random Forest ($p^*\approx p/3$)	695	257	33	290	0.70
XGBoost	468	7	30	37	–
LightGBM	467	10	23	33	–

Is each step a real improvement, or replication noise? Table 2 reports averages over the 20 training draws. An average can hide a lot: is the estimator that looks better on average reliably better, or are the 20 draws noisy enough that the ranking could flip with other draws? Every draw refits all six estimators on its own training panel and scores them on the *same* fixed test panel. The 20 replications therefore give 20 paired MSE values for each pair of estimators, and a paired t-test on the 20 differences answers the question directly. Writing $\mathrm{MSE}_A^{(r)}$ and $\mathrm{MSE}_B^{(r)}$ for the two estimators' test MSE on replication r, the test is

$$H_0 : \mathbb{E}\Big[\mathrm{MSE}_A^{(r)} - \mathrm{MSE}_B^{(r)}\Big] = 0, \qquad H_1 : \mathbb{E}\Big[\mathrm{MSE}_A^{(r)} - \mathrm{MSE}_B^{(r)}\Big] \neq 0, \tag{21}$$

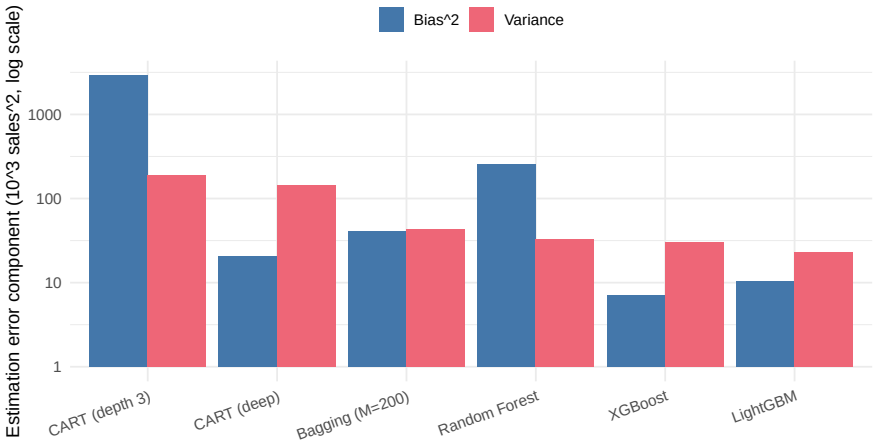


Figure 6: Bias–variance decomposition of the estimator sequence on the simulated panel (log scale). Each refinement attacks the component the previous estimator leaves largest: depth primarily reduces bias here, averaging reduces variance, feature subsampling trades variance against bias, and regularised boosting holds both down at once.

This is the same logic as the Diebold–Mariano test used later on the Rossmann hold-out (Equation (31)), but paired over training draws instead of over time. Rejecting H_0 means the average gap in Table 2 is not just luck in which 20 draws were run. Not rejecting it means the two estimators cannot be told apart on this panel with this many draws, whatever their averages say. Because all $\binom{6}{2} = 15$ pairs are tested on the same 20 draws, the p-values are Holm-corrected, so the stars below control the family-wise error rate instead of inflating it through repeated testing.

Table 3: Paired t-tests (Equation (21)) on the 20 per-replication test MSEs, all $\binom{6}{2} = 15$ estimator pairs, ordered by Holm-adjusted p-value. Mean difference is $\overline{\text{MSE}_A - \text{MSE}_B}$ in raw sales^2 units. $^{*}p < .05$, $^{**}p < .01$, $^{***}p < .001$ after Holm correction.

Comparison (A vs B)	Mean diff.	t	Holm p	Sig.
CART (depth 3) vs CART (deep)	2967899	846.02	< 0.001	***
CART (depth 3) vs LightGBM	3096506	820.17	< 0.001	***
CART (depth 3) vs XGBoost	3095189	777.93	< 0.001	***
CART (depth 3) vs Bagging	3047424	737.25	< 0.001	***
CART (depth 3) vs Random Forest	2831690	491.69	< 0.001	***
Random Forest vs XGBoost	263499	58.17	< 0.001	***
Random Forest vs LightGBM	264816	55.23	< 0.001	***
CART (deep) vs XGBoost	127290	52.10	< 0.001	***
CART (deep) vs LightGBM	128607	49.38	< 0.001	***
Bagging vs Random Forest	−215733	−48.83	< 0.001	***
CART (deep) vs Bagging	79525	45.42	< 0.001	***
Bagging vs XGBoost	47766	28.81	< 0.001	***
CART (deep) vs Random Forest	−136208	−27.02	< 0.001	***
Bagging vs LightGBM	49082	26.40	< 0.001	***
XGBoost vs LightGBM	1317	1.23	0.235	

Read against Equation (11), the sequence explains why the empirical comparison in Section 4 centres on XGBoost and LightGBM: on this panel they have the lowest estimation error. That is a reason to use them, but it does not remove tuning or representation effects from the empirical comparisons. The simulation also shows one caveat that the empirical results repeat: variance-reduction tricks are not free, and which one wins depends on how the signal is spread across the features.

3.5 Statistical Benchmark Models

The tree estimators recover $\hat{\delta}(\mathbf{x}_{s,t})$ from features shared across stores. The statistical benchmarks take the opposite route. Each is fitted to a single store's own sales history $\{y_{s,1}, \ldots, y_{s,T}\}$ with no cross-store features, so it can estimate the level μ_s and the part of $\delta_{s,t}$ that is visible in the series' own autocorrelation, but nothing that depends on the calendar or promotion variables it never sees. Three are used, each fitted separately for every store.

Seasonal ARIMA. For store s the daily series is a seasonal ARIMA process of order $(p, d, q)(P, D, Q)_7$,

$$\Phi(L)\,\Phi_7(L^7)\,(1-L)^d(1-L^7)^D\,y_{s,t} = \Theta(L)\,\Theta_7(L^7)\,u_{s,t}, \qquad u_{s,t} \sim \text{WN}(0, \sigma^2_{u,s}), \quad (22)$$

where L is the lag operator, Φ and Θ are the non-seasonal autoregressive and moving-average polynomials, Φ_7 and Θ_7 their seasonal (period-7) counterparts, and $u_{s,t}$ a white-noise innovation. This innovation belongs to the fitted ARIMA model and is not the same object as the residual in Equation (1). The orders $(p, d, q)(P, D, Q)_7$ are chosen per store by AICc. The differencing $(1-L)^d(1-L^7)^D$ removes trend and weekly seasonality, so ARIMA recovers the weekly part of $\delta_{s,t}$ through the series' own autocorrelation, not through a day-of-week feature.

Exponential smoothing (ETS). An innovations state-space model with level, trend, and weekly-seasonal states, in additive form,

$$y_{s,t} = \ell_{t-1} + b_{t-1} + c_{t-7} + u_{s,t}, \tag{23}$$

$$\ell_t = \ell_{t-1} + b_{t-1} + \alpha_\ell\, u_{s,t}, \quad b_t = b_{t-1} + \alpha_b\, u_{s,t}, \quad c_t = c_{t-7} + \alpha_c\, u_{s,t}, \tag{24}$$

where ℓ_t, b_t, c_t are the level, trend, and seasonal states, $\alpha_\ell, \alpha_b, \alpha_c$ the corresponding smoothing parameters, and $u_{s,t}$ the one-step innovation, in the same role as the ARIMA innovation in Equation (22). The error, trend, and seasonal components may each be additive, multiplicative, or absent, and the form is selected per store by AICc.

Theta. The Theta method rescales the local curvature of the series. It forms theta-lines $Z_t(\theta)$ defined by $\nabla^2 Z_t(\theta) = \theta\, \nabla^2 y_{s,t}$, where ∇^2 is the second difference, and decomposes the series at $\theta = 0$, a long-run linear trend, and $\theta = 2$, which doubles the curvature and is extrapolated by simple exponential smoothing. The forecast averages the two lines (Assimakopoulos and Nikolopoulos, 2000).

Because all three see only the store's own past, they have no access to the promotion and calendar variables that carry much of $\delta_{s,t}$.

What each method should be able to reach. The univariate benchmarks use each store's full observed history. The history-only trees of Stage 5 use a few lag summaries that are deliberately old. Both draw on the store's own past, but they represent it differently and with different recency, so a flip in the ranking between them does not identify a hidden demand component. ARIMA, ETS and Theta can pick up recurring patterns without promotion indicators, but they cannot plan for an upcoming promotion unless it is given to them as an input. This is a limit of these particular specifications, not of statistical forecasting in general: regression with ARIMA errors can take known future covariates (Hyndman and Athanasopoulos, 2021). Section 4.3.6 runs that comparison on common stores and windows, and whether the trees still win there is an empirical question.

3.6 Dataset Characteristics as Determinants of the Performance Gap

Equation (11) points to four dataset characteristics that govern the expected gain over a store-mean baseline.

1. Strength of within-store predictable variation. When the predictable swings around the store mean are small, a constant store-level forecast already captures most of the signal worth capturing. Strong seasonal or promotional patterns open up more room to improve. M5 carries weekly seasonality, events, and SNAP cycles at product level. Store-level data retain day-of-week and promotion effects, but aggregate store totals come out smoother than product-level series.

2. Feature quality and interaction richness. A model can only learn the variation $\mathbf{x}_{s,t}$ actually represents. Lags supply recent demand information. Promotion, price, and calendar variables describe what might be driving the changes. The first stage of the empirical design leaves lags out and works from calendar, promotion, and store characteristics. Stage 3 then adds lags and rolling means, so the effect of feature enrichment can be read off directly.

3. Sample scale: series count and history length. Both dimensions on which the Rossmann panel is smaller than M5 enter here. A global model pools observations across series, and M5's 30490 bottom-level series against Rossmann's 1115 stores is a difference of more than an order of magnitude. M5's roughly 5.4 years against Rossmann's 2.5 also leaves fewer repeated seasonal cycles to estimate from. Both dimensions act on the estimation-error term of Equation (11) rather than on the feature ceiling. More series and longer histories should mean more examples of recurring calendar or promotion effects and more precise seasonal estimation, but only where those effects are shared across the panel and where older observations remain relevant. Predictions 1 and 2 in Section 3.7 state these as testable expectations.

4. Naïve baseline strength. Relative improvement depends on the benchmark as much as on the model. A stable store mean leaves less to improve on. That is why the analysis reports a store-day-of-week baseline alongside the store mean. It captures weekly seasonality and sets a tougher bar.

3.7 Testable Predictions of the Framework

The framework motivates six testable predictions for the empirical experiments. They do not follow from the decomposition alone: each also assumes that its lever moves the quantity it targets, for example that stores share structure a pooled

model can learn. Each one translates a driver of Equation (11) into an *operational* lever that the experiments can move, such as the number of stores or the store-type composition. The three drivers are the predictable variation $\mathrm{Var}(\delta_{s,t})$, the feature ceiling ρ^2_∞, and the estimation error. Predictions 1 and 2 move the estimation error, Predictions 4 and 6 move potential information access, and Prediction 3 changes the composition of predictable variation. Prediction 5 is the exception: it concerns the estimators themselves rather than the data, and the simulation in Section 3.4 already says something about it. Section 5.1 returns to two cases where the lever turns out to be a poor proxy for the mechanism it was meant to track.

Prediction 1. Scale. The ensemble advantage is expected to increase with the number of stores because more series provide more opportunities for shared pattern learning.

Prediction 2. Training length. The ensemble advantage is expected to increase with the training window because longer histories provide more observations and seasonal repetitions.

Prediction 3. Heterogeneity. More homogeneous store groups are expected to produce a stronger simple baseline and a smaller ensemble advantage.

Prediction 4. Feature richness. Adding lags and rolling means is expected to improve XGBoost and LightGBM by supplying recent demand information.

Prediction 5. Method hierarchy. LightGBM, XGBoost, and Random Forest are expected to perform better than the simpler tree and linear specifications because they combine flexible interactions with ensemble regularisation. The simulation in Section 3.4 already qualifies the Random Forest half of this claim, since bagging outperforms it there once one feature carries most of the signal.

Prediction 6. Information environment. As the feature set is enriched from a store's own sales history alone to full calendar, store, and promotion information, the ensemble advantage should grow. With only its own history a boosted model is expected to do no better than the simple and statistical baselines, while richer information should make it clearly superior, mirroring the shift from the early M competitions to M5.

The fifth prediction concerns the implementations used in this study and is tested directly by the Stage 1 ranking, where the qualification noted above is expected to reappear. The sixth prediction concerns the information environment and is tested by the nested feature experiment in Stage 5.

4 Empirical Application: Rossmann Store Sales

4.1 Data

This section describes the data used in the empirical analysis. It introduces the Rossmann Store Sales dataset and its variables, reports descriptive statistics and the main empirical patterns, sets out the feature engineering and the train–test split, contrasts the dataset structurally with M5, and checks the observed properties against the assumptions behind the model of Section 3. All figures in this section are produced by `rossmann_eda.Rmd` and exported to `eda_outputs/`.

4.1.1 Overview

The Rossmann Store Sales dataset was released for a Kaggle competition in 2015 (Kaggle, 2015). The task is to forecast daily sales for 1115 Rossmann drugstores from roughly 2.5 years of observations, 1 January 2013 to 31 July 2015. Unlike M5, the target is total daily sales for each store instead of sales of individual products, so the problem is smaller and more aggregated.

The raw training file records every store–day, including days on which a store is closed. Removing closed days (`Open` $= 0$) and the small number of open days with zero recorded sales leaves the modelling sample of approximately 844,000 open store–days used throughout. Closed days are excluded because the analysis models realised sales on open days. Retaining them would also lower the store-mean benchmark mechanically and leave the percentage error (MAPE) undefined.

Review of published competition code. The Rossmann competition was won by tree models. Bojer and Meldgaard (2021) review its top solutions: the winner used an ensemble of global XGBoost models, most other top performers also used ensembles of global XGBoost models, and the third place was, among the competitions they review, the first time a neural network reached the top three. The winner did not win through the algorithm alone. Most of the gain came from features: sales statistics and their moving averages at different levels, for different days of the week and promotion periods, counters of the days before, during and after holidays and promotions, and calendar indicators. This is the same point this thesis makes with its own experiments, that the trees win through the information they are given. The third-place Rossmann solution is public, and its competition-specific `kaggle` branch was inspected (Guo, 2015). Its `prepare_features.py` drops zero-sales training rows, leaves out `Customers`, builds promotion and competition-duration features, and adds outside data on weather, regions and search trends. The README describes learned category embeddings and averaging over several networks. This is a useful check on the feature choices made here, but not an accuracy benchmark to compare against, because the information, the model family and the validation setup are all different. This thesis keeps a smaller, documented feature set so that its

comparisons stay easy to interpret. Outside data about the future, such as weather, would also need a check that it is really available at the forecast origin before it could be used in practice.

4.1.2 Variables

The data come in two files. The training file holds daily observations. The supplementary store file adds time-invariant store characteristics. Table 4 lists the variables used.

Table 4: Variables in the Rossmann data.

Variable	Type	Description
Store	Integer	Unique store identifier (1–1115)
DayOfWeek	Integer	Day of week (1 = Monday, 7 = Sunday)
Date	Date	Calendar date
Sales	Integer	Daily turnover in EUR (target variable)
Customers	Integer	Number of customers (excluded: unknown at forecast time)
Open	Binary	Whether the store was open (0/1)
Promo	Binary	Whether a promotion was running (0/1)
StateHoliday	Factor	State holiday type (none, public, Easter, Christmas)
SchoolHoliday	Binary	Whether schools were on holiday (0/1)
StoreType	Factor	Store format (A, B, C, D)
Assortment	Factor	Assortment level (basic, extra, extended)
CompetitionDistance	Numeric	Distance to nearest competitor (metres)
CompetitionOpenSince	Date	Date the nearest competitor opened
Promo2	Binary	Participation in a recurring promotion scheme (0/1)

4.1.3 Data Preparation

Every script in the replication folder applies the same preparation, so that a result never depends on which file produced it. The steps are few and are set out here in full.

Row filters. The raw file has 1,017,209 rows. Removing the 172,817 closed-day rows leaves 844,392, and removing the 54 open days with zero sales leaves 844,338. The data do not say why those 54 days are zero. Dropping them gives a clean target, sales on open days with positive sales, and keeps MAPE defined.

Joining and encoding. The store file is joined on `Store`. The archived tree models code `StoreType`, `Assortment` and `StateHoliday` as integers. This gives the categories an order they do not really have: a single split can only separate, say, types 1–2 from 3–4, not types 1 and 3 from 2 and 4. Several splits can partly work around this, but with limited depth the coding still matters, so it is not the same as one-hot or categorical treatment. The ARIMAX comparison in Section 4.3.6 uses explicit indicators instead, including store indicators for the trees.

Missing values. Three stores have no `CompetitionDistance`. The archived scripts fill it with the median over the joined rows. That is a median weighted by rows, not the median over stores, and because it is computed before the train–test split it uses the row mix of later periods, although no test sales enter it. Filling it from the training data only would be cleaner; the archived results keep this limitation. The opening date of the nearest competitor is missing for 354 stores. `CompetitionOpenMonths` counts the months since that opening, floored at zero, and stores with an unknown date also get zero. An unknown date is therefore treated like a competitor that has just opened, not like one that has always been there. Because every store with a missing date gets the same value, the trees can partly tell that the date was missing; an explicit missing-value indicator or a different coding would still be a useful robustness check.

Calendar alignment. The main tree analyses keep only open days with positive sales, so their lag operators count retained rows, not calendar days. The statistical benchmarks need a regular daily calendar, so for them closed days are filled with zero. The ARIMAX comparison trains every model on the same daily calendar and scores them on the same open days with positive sales.

What is not done. No sales value is trimmed, winsorised or transformed. This is a decision, not an oversight: the distribution has a long right tail, as the skewness in Table 5 shows, and the largest days sit far above the median.

Those days are kept because they are the phenomenon under study. A sales figure several times a store's usual level is typically a promotion, a pre-holiday surge, or the reopening after a closure, and all three are exactly the predictable within-store variation $\delta_{s,t}$ that the thesis asks whether a model can recover. Removing them would remove the signal and leave an easier problem on which every method looks better than it is. A trimmed sample would also flatter the store-mean baseline, which is most exposed to the tail, and so would narrow the very gap being measured. Dropping the 54 zero-sales open days is a choice about the target, not a claim that those days are recording errors; the data cannot tell.

The raw euro scale is kept for a related reason: RMSE is then read in the units of the quantity being forecast, and a log transform would change what the loss function

penalises, weighting proportional errors on small stores as heavily as absolute errors on large ones.

The derived calendar and store features are listed separately in Section 4.1.5, and `Customers` is excluded from every model for the reason given there.

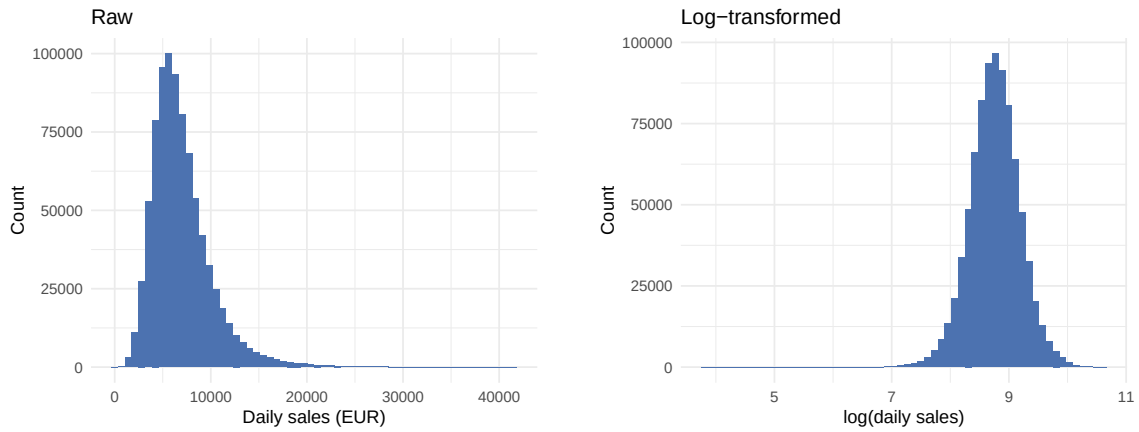
4.1.4 Descriptive Statistics

Table 5 summarises the distribution of the target across all open trading days. Daily store sales average about 6960 euros, with a standard deviation near 3100 and a pronounced right skew.

Table 5: Descriptive statistics: daily store sales (EUR), open days only.

Statistic	Value
Mean	6956.0
Median	6369.0
Std. Dev.	3103.8
Min	46.0
Max	41551.0
Skewness	1.59

The right skew in the raw target (Figure 7, raw panel) means that a few large errors weigh heavily in RMSE. The skew by itself does not favour any forecasting family. Trees make no assumption about the distribution of the target, and the log panel is there only to make the skew visible.



(a) Raw

(b) Log-transformed

Figure 7: Distribution of daily sales.

Aggregating sales by month over the full sample (Figure 8) reveals the overall trend and a clear yearly cycle, with the December peak standing out.

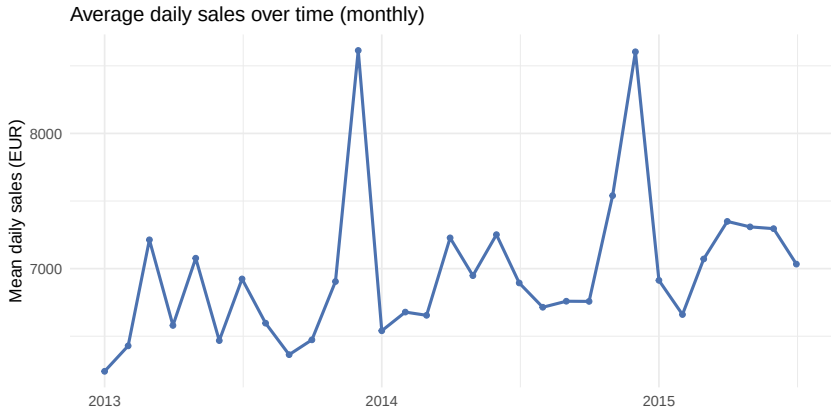
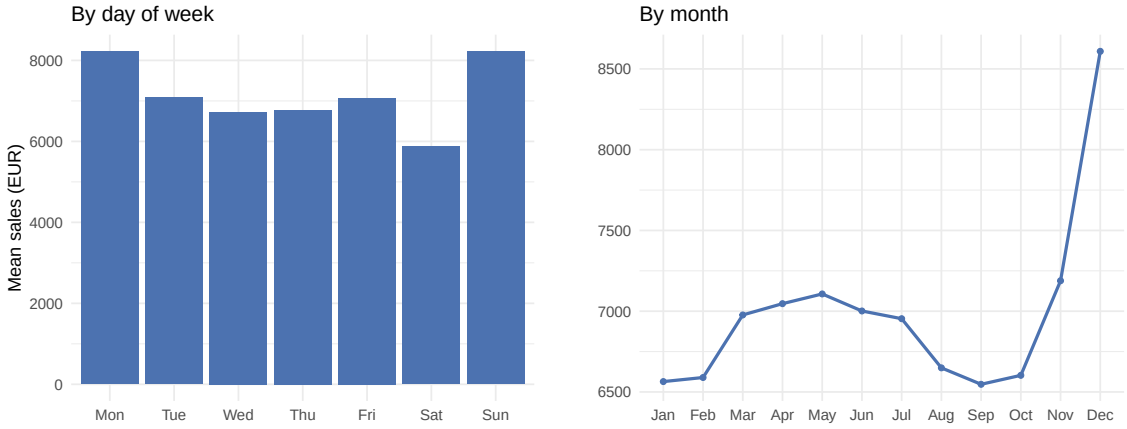


Figure 8: Average daily sales over time (all stores, monthly aggregation).

The two seasonal patterns are shown directly in Figure 9. Sales peak on Mondays and are lowest on Saturdays. Most stores are closed on Sundays; the few that open, mostly Type B, have high sales, so the Sunday bar reflects those stores rather than a typical Sunday. The monthly profile shows the pronounced December effect consistent with Christmas shopping. This strong, repeating structure is the predictable component $\delta_{s,t}$ that a store mean cannot capture but a day-of-week baseline and the trees can.



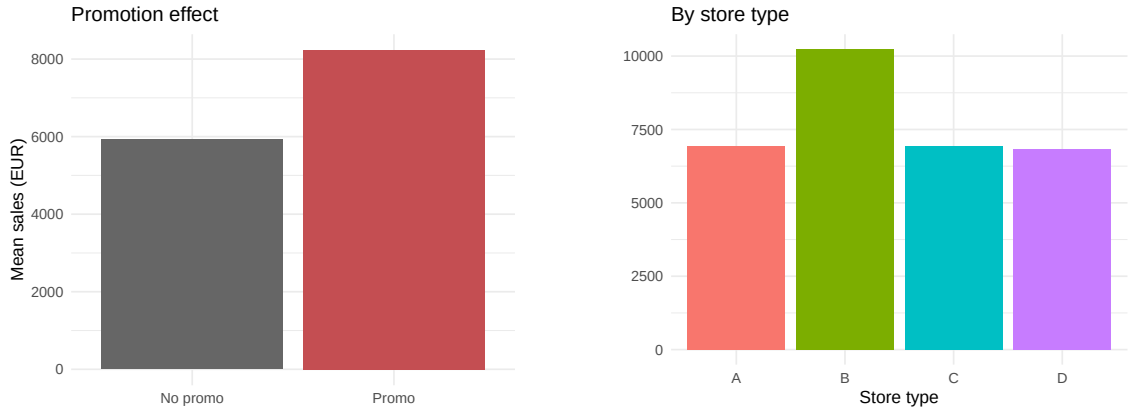
(a) By day of week

(b) By month

Figure 9: Within-week and within-year seasonality of open-day sales.

Promotions and store format are the other two strong drivers (Figure 10). Promotion days carry markedly higher average sales, and Type B stores stand apart from the rest.

The four store types differ in average level, in opening policy, and in how much of their variation is weekly. Table 6 summarises them. Type B stores are open on almost every Sunday, record average sales well above the other types, and carry the largest weekly-predictable share, the proportion of within-store variance explained by store-specific day-of-week means. This makes them the most predictable group



(a) Promotion effect

(b) By store type

Figure 10: Average sales by promotion status and store type.

and is what drives the store-type experiment in Section 4.2.4.

Table 6: Store types in the Rossmann data. Sunday opening is the share of Sundays on which stores of the type are open. Mean sales are open-day averages. The weekly-predictable share is the proportion of within-store sales variance explained by store-specific day-of-week means, averaged across stores over the descriptive full sample.

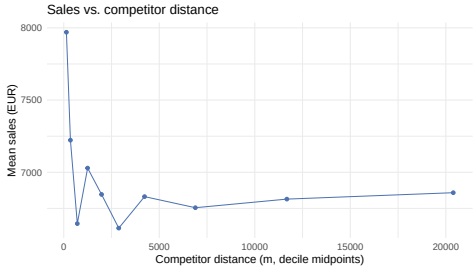
Store type	Stores	Sunday opening (%)	Mean open-day sales (EUR)	Weekly-predictable share
A	602	1.4	6913	0.29
B	17	98.4	10112	0.39
C	148	0.0	6918	0.16
D	348	0.6	6824	0.15

Among the continuous store attributes, distance to the nearest competitor has only a weak relationship with average sales (Figure 11, first panel), and the numeric features are weakly correlated with one another (Figure 11, second panel), which is consistent with the irregular, low-correlation feature space on which tree ensembles tend to do well (Grinsztajn et al., 2022).

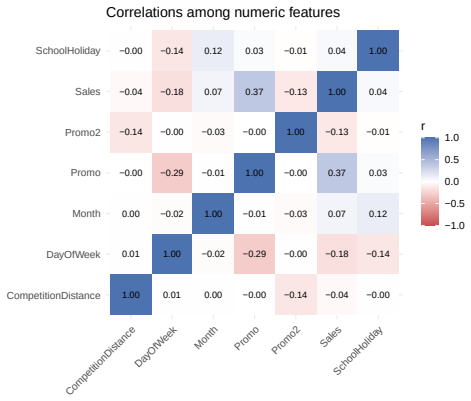
At the level of a single store, Figure 12 shows the full daily history and the promotion effect for a representative store, illustrating the roughly stable level and the recurring promotion-driven spikes that the panel patterns above aggregate.

4.1.5 Feature Engineering

Raw date and store fields are turned into a set of model-ready features used by all tree-based models, listed in Table 7. The variable `Customers` is excluded from every model because it is not available at forecast time. A practitioner predicting tomorrow's sales does not yet know how many customers will visit, so including it would constitute data leakage.

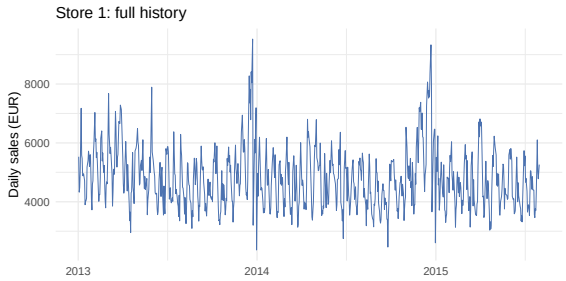


(a) Sales vs. competitor distance

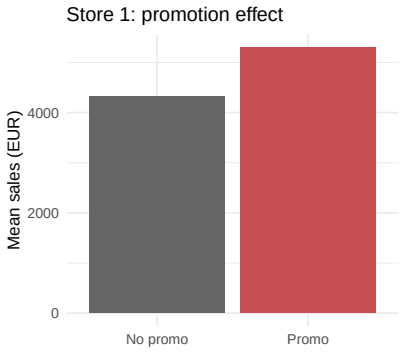


(b) Feature correlations

Figure 11: Competitor distance and correlations among the numeric features.



(a) Full sales history



(b) Promotion effect

Figure 12: Store 1: full daily sales history and average promotion effect.

Table 7: Engineered features used in all tree-based models.

Feature	Description
Store	Store ID (numeric)
DayOfWeek	Day of week (1–7)
Promo	Promotion active (0/1)
StateHoliday	State holiday type coded 0–3 (none, public, Easter, Christmas)
SchoolHoliday	School holiday indicator (0/1)
StoreType	Store format encoded as integer (1–4)
Assortment	Assortment level encoded as integer (1–3)
CompetitionDistance	Distance to nearest competitor (metres)
CompetitionOpenMonths	Months since nearest competitor opened
Promo2	Participating in recurring promo scheme (0/1)
Year, Month, Week	Calendar year, month (1–12), ISO week (1–53)
DayOfMonth, Quarter	Day of month (1–31), calendar quarter (1–4)
IsWeekend	Saturday or Sunday indicator (0/1)

4.1.6 Train–Test Split

All models are evaluated on a temporal hold-out: the final six weeks (42 days), from 20 June to 31 July 2015, form the test set, and all earlier observations form the training set, a span chosen because six complete weeks give every day of the week equal representation. A temporal split mirrors real forecasting practice and is necessary to avoid leaking future information into training. A random split would let a model interpolate within the test period rather than extrapolate beyond the training data. Stages 1–3 and 5 keep this test period fixed to isolate changes in training conditions and features, while Stage 4 moves the forecast origin to check robustness across windows.

4.1.7 Comparison with the M5 Dataset

Several structural differences between Rossmann and M5 bear directly on the model of Section 3 and motivate using Rossmann as an external test of the M5 result. Table 8 summarises them.

Table 8: Structural comparison of the Rossmann and M5 datasets.

Property	Rossmann	M5 (Walmart)
Country	Germany	United States
Retailer type	Drugstore chain	General merchandise
Bottom-level series	1115 stores	30490 product–store
Target	Aggregate store sales (EUR)	Units per product–store
History	≈2.5 years	≈5.4 years
Frequency	Daily	Daily
Intermittency	Low (aggregated)	High (product level)
Prices	Not observed	Product-level prices
Calendar/promotion	Promotions, holidays	Events, SNAP, prices

The differences pull in a consistent direction. Rossmann offers a global model far fewer related series, a shorter history with fewer seasonal repetitions, and no product-level prices, while its aggregation makes sales smoother and less intermittent, which should strengthen the simple store-level baselines. If LightGBM's M5 performance leaned partly on the scale and feature richness of that dataset, its advantage need not survive here.

4.1.8 Properties versus Model Assumptions

The framework treats each store's level as roughly stable within an evaluation window. The exploratory plots show recurring weekly and promotion patterns, but also trends and level shifts that strain this assumption. Table 6 reports the average within-store weekly share on open trading days: 0.29 for Type A and 0.39

for Type B. These are descriptive numbers over the full sample, not training-only estimates of all predictable variation. A pooled variance ratio that also counts differences between store levels would answer a different question, so it is not used in their place. The calendar-aligned STL plot in Figure 13 is only an illustration: its interpolated closed days and smoothing choices mean it cannot identify the hidden components of Equation (1).

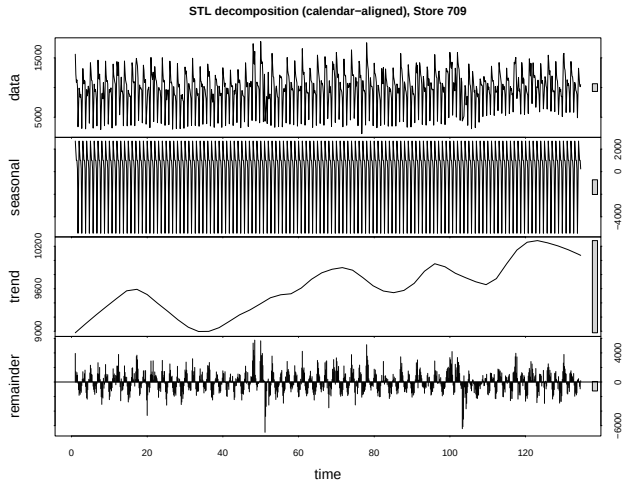


Figure 13: Calendar-aligned STL decomposition of one store's daily series into trend, weekly seasonal, and remainder components. Closed days are filled by linear interpolation so the weekly cycle stays aligned to actual days of the week.

The next subsection sets out the experiments.

4.2 Research Design

4.2.1 Overview

The empirical analysis has five main stages, followed by a set of additional analyses (Stage 6, Section 4.3.7) that probe the internal validity of the results. Section 4.3.6 adds one more check: it gives ARIMAX the same covariates as the trees, to see whether the tree advantage comes from the extra information rather than from the trees themselves.

Stage 1: Full Rossmann comparison. Eleven specifications meet on the final 42-day holdout: a store-mean baseline, a store-day-of-week baseline, Auto-ARIMA, ETS, Theta, linear regression, CART, bagging, Random Forest, XGBoost, and LightGBM. This stage fixes the initial ranking without lag features.

Stage 2: Subsetting experiments. The analysis changes the number of stores, the length of the training window, and the store-type composition. XGBoost, LightGBM, and both simple baselines are evaluated in each condition. This stage tests the scale, training-length, and heterogeneity expectations.

Stage 3: Feature enrichment. Sales lags and rolling means are added, after which XGBoost and LightGBM are re-estimated on the full dataset. The comparison shows how much recent sales information changes their performance.

Stage 4: Rolling test-window robustness. The no-lag comparison is repeated over four 42-day periods. This checks whether the main XGBoost–LightGBM ranking depends on the final holdout window (Ma and Fildes, 2022).

Stage 5: Incremental information sets. XGBoost and LightGBM are re-estimated on four nested feature sets, starting from a store's own sales history and adding calendar, store, and promotion information in turn. All features are available at the forecast origin, so this stage provides a fixed-origin counterpart to Stage 3 and tests the information-environment explanation developed in the literature review.

Stage 6: Additional analyses. A set of robustness and internal-validity checks on the main results: fixed-origin recursive lag forecasts, Diebold–Mariano significance tests, replication standard errors for the number-of-stores experiment, a direct store-level test of the framework, a hyperparameter-capacity grid, and the fixed-composition and global-versus-local comparisons. These do not introduce new conditions. They probe how robust the first five stages are.

The stages are operational designs, but each is included because it targets a data property from the framework. Table 9 makes this mapping explicit. The aim is to avoid interpreting a Rossmann-specific lever, such as the number of stores, as though it were the general principle itself.

Table 9: Experimental levers and the data properties they target.

Experiment	Operational variation	Data property targeted
Stage 1	Full-sample model ranking	Whether flexible estimators recover predictable structure beyond simple baselines
Stage 2, store count	50, 200, 500, and 1115 stores	Cross-series learning potential and composition-induced heterogeneity
Stage 2, training length	6, 12, 18, 24 months, and full history	Seasonal repetition and estimation precision
Stage 2, store type	A, B, C+D, and full sample	Baseline strength and predictable within-store variation
Stage 3	No lags versus lag and rolling features	Recoverability of recent demand structure by the feature set
Stage 4	Four 42-day test windows	Dependence of the measured advantage on the forecast origin
Stage 5	History, calendar, store, and promotion feature sets	Observable signal richness, corresponding to the feature ceiling ρ^2_∞

4.2.2 Models

Eleven specifications meet on the Stage 1 holdout. Their mechanics are set out in Sections 3.4 (tree-based) and 3.5 (statistical). Table 10 records only the estimation settings used here.

Table 10: Stage 1 specifications and their estimation settings. Mechanics are described in Section 3.4 for the tree-based estimators and Section 3.5 for the statistical benchmarks. Early stopping uses the last 15% of training dates as a temporal validation slice.

Specification	Estimation settings
Store-mean	$\hat{y}_{s,t} = \bar{y}_s$, mean open-day training sales per store
Store-day-of-week	mean training sales per store × day of week
Auto-ARIMA	Equation (22), per store via `forecast`; orders by AICc, stepwise approximate search on the weekly-frequency series
ETS	Equations (23)–(24), per store via `forecast::ets`; error, trend, seasonal forms by AICc
Theta	Section 3.5, per store via `forecast::thetaf`
Linear regression	OLS on the numeric Stage 1 features, store ID included as a number; a simple reference, not a well-specified linear model
CART	`rpart`; `maxdepth` 20, `cp` 0.00001, `minsplit` 10
Bagging	`ranger`; $M = 200$ deep trees, `mtry` $= p$ (every split may use all predictors)
Random Forest	`ranger`; $M = 200$ deep trees, `mtry` $\approx p/3$ (regression default)
XGBoost	`xgboost`; max depth 6, $\eta = 0.10$, rounds by early stopping, subsample 0.8, colsample 0.8
LightGBM	`lightgbm`; `num_leaves` $= 63$, $\eta = 0.05$, rounds by early stopping, subsample 0.8, colsample 0.8

Bagging and the Random Forest differ only in `mtry`. LightGBM's leaf-wise 63-leaf growth is a different capacity profile from the depth-6 XGBoost model.

For ARIMA, ETS, and Theta, closed days are retained as zero-sales observations when constructing each daily series so that weekly spacing is not distorted. A closed day is not literally a zero-sales day, so this could in principle bias the level and weekly seasonal of these models downward. A robustness check that instead fills closed days by interpolation does not bear this out. It leaves Auto-ARIMA and ETS slightly worse (by about 2% and 8%) and improves only Theta (by about 11%), so the zero-fill is not what places the statistical benchmarks behind the boosted trees. Their accuracy is in any case evaluated only on the open-store test observations used for all other models.

The models are run at standard configurations instead of each being grid-tuned to its best. The tree depths differ by design, since the two boosted models add many trees and so use shallow ones (depth six and 63 leaves) with the number of rounds chosen by early stopping on a temporal validation slice, while CART, bagging, and the random forest average or stand on deep trees. The comparison is therefore between sensible default specifications, and Stage 6 probes the sensitivity of the boosted models to their capacity settings directly.

4.2.3 Evaluation Metrics

RMSE is the primary ranking metric, while MAE and MAPE provide additional views of forecast error. M5 used WRMSSE, which scales and weights product-level series within a hierarchy. Rossmann contains aggregate store sales and does not have the same hierarchy. Numerical improvements in M5 and Rossmann are therefore not directly comparable. The relevant comparison is whether the broad model ordering carries over.

All metrics are computed over $\mathcal{S}$, the set of test-period store–day pairs, with $n = |\mathcal{S}|$, $y_{s,t}$ observed sales as in Equation (1), and $\hat{y}_{s,t}$ the model's prediction:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{(s,t)\in\mathcal{S}} (y_{s,t} - \hat{y}_{s,t})^2}, \qquad \text{MAE} = \frac{1}{n}\sum_{(s,t)\in\mathcal{S}} |y_{s,t} - \hat{y}_{s,t}|\,, \tag{25}$$

$$\text{MAPE} = \frac{100\%}{n}\sum_{(s,t)\in\mathcal{S}} \left|\frac{y_{s,t} - \hat{y}_{s,t}}{y_{s,t}}\right|, \qquad \Delta_{\text{rel}} = \frac{\text{RMSE}^{\text{naive}} - \text{RMSE}^{\text{model}}}{\text{RMSE}^{\text{naive}}} \times 100\%. \tag{26}$$

RMSE, in euros, weights large errors most heavily and is the ranking metric. MAPE is undefined when $y_{s,t} = 0$, a further reason closed-store observations are excluded. Positive Δ_{rel} means lower RMSE than the store-mean baseline. Improvement over Store-DOW is also reported for the main ensemble models.

4.2.4 Stage 2: Subsetting Experiment Design

The subsetting experiments vary one aspect of the Rossmann sample while retaining the final 42-day test period. XGBoost, LightGBM, Store-mean, and Store-DOW are recalculated for each condition.

Dimension 1: Number of stores. Stores are sampled without replacement to create subsets of $N \in \{50, 200, 500, 1115\}$. The first three conditions use 20 random draws. The full condition uses all stores once. Averaging across draws reduces the influence of any particular small-store sample.

Dimension 2: Training window length. The training sample retains the most recent 6, 12, 18, or 24 months, or the full available history. The test period remains unchanged.

Dimension 3: Store type composition. The store-type conditions are A alone, B alone, the balanced C+D mix, and the full sample. The C+D mix replaces a Type-A-plus-B grouping that would have been dominated by Type A, since Type B has only 17 stores. Type B is still reported on its own but interpreted cautiously for the same reason.

4.2.5 Stage 3: Feature Enrichment (Phase 2)

Stage 3 adds sales lags and rolling means to XGBoost and LightGBM. For store s on day t, the lag variables are:

$$\text{lag}_k(s,t) = y_{s,\,t-k}, \quad k \in \{1, 7, 14, 28\} \tag{27}$$

Here, $k = 1$ denotes the previous retained trading observation and $k = 7$ seven retained trading observations earlier. The rolling means are:

$$\text{roll}_7(s,t) = \frac{1}{7}\sum_{k=1}^{7} y_{s,\,t-k} \tag{28}$$

$$\text{roll}_{28}(s,t) = \frac{1}{28}\sum_{k=1}^{28} y_{s,\,t-k} \tag{29}$$

Here t counts retained trading days, not calendar days, so seven or 28 steps back is usually a little more than one or four calendar weeks. The rolling means use only earlier observations. During the holdout, lag variables are updated as actual sales become available. Stage 3 is therefore a rolling one-day-ahead evaluation, not a fixed-origin 42-day forecast.

The comparison tests whether recent sales information improves the two boosted models and whether it narrows their Stage 1 performance gap.

4.2.6 Stage 4: Rolling Test-Window Robustness

The no-lag comparison is repeated for four 42-day windows. For each window, training uses all observations before the first test date. Store-mean, Store-DOW, XGBoost, and LightGBM are then re-estimated.

Using the no-lag feature set keeps test-period sensitivity separate from the lag experiment. The ranking is considered stable across these windows if the same boosted model has lower RMSE in each period.

4.2.7 Stage 5: Incremental Information Sets

The literature review argued that the change in machine-learning performance between the early M competitions and M5 reflects the information environment as much as the algorithms. Stage 5 turns this argument into an experiment within

Rossmann. XGBoost and LightGBM are re-estimated on four nested feature sets that move step by step from an old-M-style setting to the full Rossmann feature set:

1. **History only:** store identity plus origin-safe summaries of the store's own sales history (lags of 42, 49, 56, and 84 retained trading observations and pre-origin rolling means over 7 and 28 days; 7 features). This approximates the univariate information available to participants in the early M competitions.

2. + **Calendar:** day-of-week, month, week, day-of-month, quarter, weekend indicator, year, and a trend term (15 features).

3. + **Store attributes:** store type, assortment, competitor distance, and competition history (19 features).

4. + **Promotions and holidays:** promotion indicators, Promo2, and state and school holidays (23 features).

The stores, training period, 42-day test period, model settings, and metrics are identical across the four sets, so performance changes can be attributed to the added information. Unlike Stage 3, no realised sales from the test period enter any predictor. The shortest lag is 42 retained rows, and a 42-day holdout has at most 42 retained rows per store, so every lag value is already known at the origin. These values can be much older than the latest observation ARIMA uses, so this comparison depends on recency and on how the history is represented, not only on the model family. Store-mean and Store-DOW are recomputed inside the experiment as fixed reference points. The warm-up rows dropped to build the origin-safe features lower them slightly compared with Stage 1 (Store-DOW 1647 against 1665).

4.3 Results

The results follow the five stages of the research design, then the covariate-matched ARIMAX check and the additional Stage 6 analyses. Δ_{rel} is the improvement over Store-mean and Δ_{DOW} the improvement over the stronger Store-DOW benchmark. The stages follow the framework; they are not a set of unrelated model comparisons. Stage 1 asks whether the boosted estimators recover more predictable structure than the baselines on the full Rossmann sample. Stages 2–5 then change empirical counterparts of $\text{Var}(\delta)$ and ρ_∞^2: the amount of recurring structure, the history available to estimate it, the store mix and the information in the feature set. Section 4.3.8 finally replaces the Rossmann stage labels with store-level data properties, so the answer to the research question is not tied only to the number of stores or months in this particular dataset.

4.3.1 Stage 1: Full-Dataset Baseline Comparison

Table 11 reports the full-sample comparison without lag features, ordered by RMSE.

Table 11: Stage 1 results: full Rossmann dataset, no lag features. Δ_{rel} is defined as $(RMSE_{\text{naïve}} - RMSE_{\text{model}})/RMSE_{\text{naïve}} \times 100\%$; positive values indicate improvement over the store-mean naïve baseline. Δ_{DOW} reports improvement over the store-day-of-week naïve baseline.

Model	RMSE	MAE	MAPE (%)	Δ_{rel} (%)	Δ_{DOW} (%)
XGBoost	1003	701	10.8	+48.2	+39.8
LightGBM	1013	723	11.6	+47.6	+39.2
Bagging	1087	743	11.4	+43.8	+34.7
Random Forest	1212	839	13.3	+37.3	+27.2
CART	1445	979	14.7	+25.3	+13.2
Auto-ARIMA	1508	1133	17.2	+22.0	+9.5
Theta	1618	1245	18.5	+16.3	+2.8
Store-DOW naïve	1665	1254	18.5	+13.9	–
ETS	1709	1296	19.8	+11.6	−2.6
Store-mean naïve	1934	1457	23.1	–	–
Linear	2693	1971	33.6	−39.2	−61.7

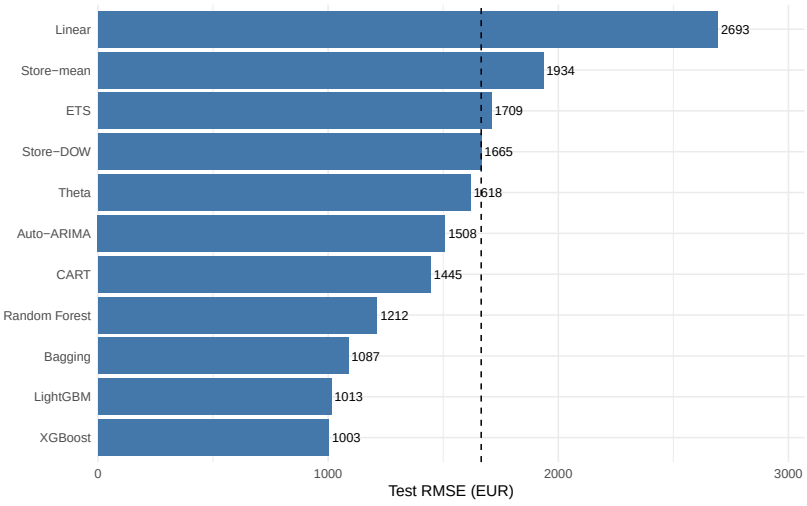


Figure 14: Stage 1 RMSE comparison on the full Rossmann dataset without lag features, ordered by RMSE. The dashed line marks the Store-DOW baseline.

XGBoost, LightGBM, bagging, and Random Forest all outperform both baselines, but the baseline comparison is itself informative. Store-DOW reduces RMSE from 1934 to 1665, showing that weekly seasonality accounts for a meaningful part of the forecast error before any flexible model is introduced. Relative to this stronger

benchmark, XGBoost improves RMSE by 39.8%, LightGBM by 39.2%, and Random Forest by 27.2%, so the ensemble advantage cannot be attributed only to a weak store-mean comparison. The trees also recover predictable structure beyond the weekly pattern.

One ordering inside the tree family stands out. Bagging and the random forest are the same bootstrap-averaged ensemble at two feature-subsampling rates, bagging letting every split use all p predictors and the random forest a random subset of $p^* \approx p/3$. On this data the gain over the store mean is larger when every feature is available at each split, so bagging (RMSE 1087) sits just behind the two boosted models and a little ahead of the random forest (1212). With the forecast origin fixed at the final 42 days, letting each split see only a few features hides part of the calendar structure the test window depends on. Feature subsampling usually helps by decorrelating the trees, but here it costs more than it saves. This is the same trade-off the simulation in Section 3.4 shows: subsampling lowered the correlation $\hat{\kappa}$ between trees but raised the bias enough to leave the random forest behind bagging (Table 2). Whether subsampling pays off depends on the data, a point the scale and global–local results come back to.

Comparison with Dedicated Statistical Models. All three statistical models were fitted separately for each of the 1115 stores and evaluated on the same holdout observations as the Stage 1 models, and no store-level fit failed. Auto-ARIMA is the strongest of the three, reaching an RMSE of 1508 and a 9.5% improvement over Store-DOW, while Theta records 1618 and a 2.8% improvement and ETS reaches 1709, which is worse than Store-DOW although still better than Store-mean. These are real forecasting models rather than elementary averages, so they raise the bar considerably. Both boosted models still clear it. XGBoost and LightGBM outperform all three on the primary holdout, and even the Random Forest beats Auto-ARIMA by a wide margin (1212 against 1508). The strength of the ARIMA result matters for the argument all the same, because it shows that the ensemble advantage is not automatic. A well-specified store-level statistical model still beats the unpenalised linear regression and comes close to a single CART tree.

Within the boosted pair the margin is much narrower. XGBoost records an RMSE of 1003 against LightGBM's 1013, a difference of about one percent. That gap is better read as a comparison between two fitted specifications than as a stable ranking of the algorithms themselves, and Stage 3 shows how even that ordering shifts once recent sales information becomes available. These results support Prediction 5 only in part. The two boosted models do lead, but third place goes to plain bagging rather than to the random forest, so the ensemble without feature subsampling ends up ahead of the one with it, and the ranking does not line up neatly with model complexity. Still, flexible tree ensembles dominate the linear specification whether or not they are regularised, and tree structure alone already captures part of Rossmann's predictable variation, since even a single deep CART tree improves

on Store-mean.

Estimated store levels, fitted departures and forecast errors. Equation (1) splits daily sales into a store level, a predictable departure and a residual. On the Stage 1 holdout their fitted counterparts can be illustrated numerically. The store level $\hat{\mu}_s$ is the store's training mean. It averages 6938 euros, with a standard deviation of 2385 euros across stores, so the stores differ a lot in level. On a test day, sales depart from this level by 1934 euros on average (root mean square), which is exactly the error of Store-mean. How much of that departure is δ depends on the information set (Section 3). For a forecast $\hat{y}_{s,t}$ that uses a given set, $\hat{\delta}_{s,t} = \hat{y}_{s,t} - \hat{\mu}_s$ and $\hat{u}_{s,t} = y_{s,t} - \hat{y}_{s,t}$, so $y_{s,t} - \hat{\mu}_s = \hat{\delta}_{s,t} + \hat{u}_{s,t}$ holds for every store-day. Table 12 reports both parts for three sets. The weekly pattern alone explains 26% of the squared departure. Calendar, store and promotion features raise this to 73%, and recent sales to 82%. These are realised reductions in squared forecast error on the holdout, not estimates of the latent parts of Equation (1). The last row also updates recent sales during the test period, so it answers a different forecasting question from the first two. Figure 15 shows the three parts for one store.

Table 12: Fitted prediction decomposition and test MSE reductions on the Stage 1 holdout (euros, root mean square over all test store-days). $\hat{\delta}$ is the forecast minus the store's training mean and $\hat{u}$ the actual sales minus the forecast. The MSE reduction is $1 - \text{mean}(\hat{u}^2)/\text{mean}((y - \hat{\mu})^2)$, with Store-mean and all three forecasts scored on the same store-days. Forecasts from the Store-DOW baseline and from LightGBM as in Stages 1 and 3.

Information set	$\hat{\delta}$	$\hat{u}$	MSE reduction vs Store-mean (%)
Weekly (Store-DOW)	981	1665	25.9
Calendar, store, promotion (LightGBM)	1626	1013	72.6
Plus recent sales (LightGBM, lags)	1765	826	81.8

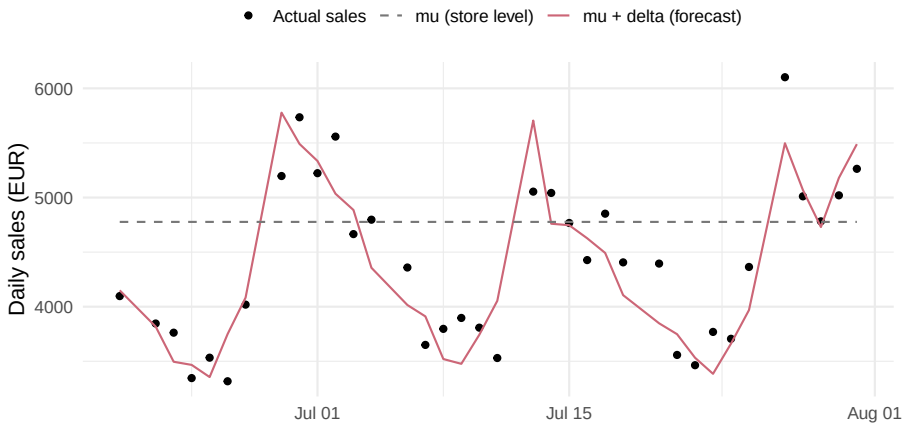


Figure 15: Store 1 on the holdout. The dashed line is $\hat{\mu}_s$, the line is the forecast $\hat{\mu}_s + \hat{\delta}_{s,t}$ (LightGBM with recent sales), and the points are actual sales. The gap between a point and the line is $\hat{u}_{s,t}$.

4.3.2 Stage 2: Subsetting Experiments

The full-sample rows in Tables 13, 14 and 15 fit the same models to the same data, so they give the same results. They differ slightly from Stage 1 (Table 11), which was run in a separate script.

Dimension 1: Number of Stores. Table 13 reports selected diagnostics for the number-of-stores experiment. Results for 50, 200, and 500 stores are averaged over 20 random draws. The full condition uses all 1115 stores once.

Table 13: Stage 2 results: number-of-stores experiment. Δ_{DOW} is the gain of the store-day-of-week baseline over the store-mean baseline. The boosted-model gains for $N \leq 500$ are means over 20 random draws, with the standard error of the mean in parentheses; the full sample is the population, so it is run once and has no sampling error.

N	Naïve RMSE	Store-DOW RMSE	Store-DOW Δ (%)	XGBoost Δ_{rel} (%)	LightGBM Δ_{rel} (%)
50	1917	1620	15.5	54.2 (0.8)	54.1 (0.9)
200	1917	1644	14.2	51.1 (0.8)	52.3 (0.8)
500	1932	1659	14.1	49.5 (0.6)	51.6 (0.6)
1115	1934	1665	13.9	47.7	47.6

Prediction 1 is rejected. XGBoost's gain falls from 54.2% at 50 stores to 47.7% in the full panel, while LightGBM falls from 54.1% to 47.6%. Store-DOW remains about 14% better than Store-mean across the four conditions. The observed relationship is the reverse of the predicted monotonic increase, and the decline is significant. Within a draw the store samples are nested, so the tests pair by draw. Regressing the gain on the logarithm of the number of stores with draw fixed effects gives a

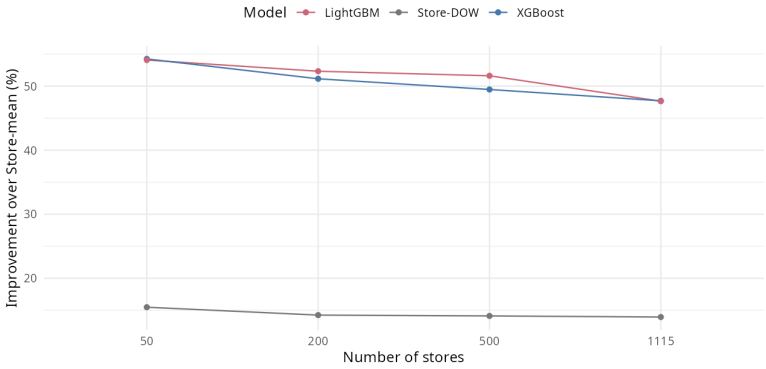


Figure 16: Stage 2 number-of-stores experiment: improvement over the store-mean baseline for the Store-DOW naïve benchmark and the two boosted models.

slope of -2.09 (standard error 0.40, $p < 0.001$) for XGBoost and -1.09 (standard error 0.38, $p = 0.007$) for LightGBM, and a paired test of 500 against 50 stores gives the same result ($p < 0.001$ and $p = 0.016$). The predicted increase is therefore contradicted in this mixed-store subsampling design. Because the evaluated stores change together with the training pool, this does not show that adding training stores itself makes forecasts worse. The experiment shows the pattern but cannot say how much of it comes from the changing store mix and how much from baseline strength. Section 4.3.7 repeats it with the store type held fixed, although stores of the same type still differ in other ways.

Dimension 2: Training Window Length. Table 14 reports Δ_{rel} as the training window is progressively extended.

Table 14: Stage 2 results: Δ_{rel} by training window length.

Window	Store-DOW Δ (%)	XGBoost Δ_{rel} (%)	LightGBM Δ_{rel} (%)
6 months	14.3	−14.4	36.8
12 months	14.9	47.3	49.9
18 months	14.4	44.8	47.7
24 months	14.4	45.4	48.9
Full (30 mo.)	13.9	47.7	47.6

Prediction 2 is supported only at the lower end of the training range. Both models do worst with six months of training data, and from 12 months onward the gains are stable, with XGBoost between 44.8% and 47.7% and LightGBM between 47.6% and 49.9%. The largest improvement comes between six and 12 months. Adding further history does not give the predicted steady increase.

The six-month window shows why one full year matters. LightGBM still improves on Store-mean by 36.8%, but XGBoost reaches −14.4%, against a gap of

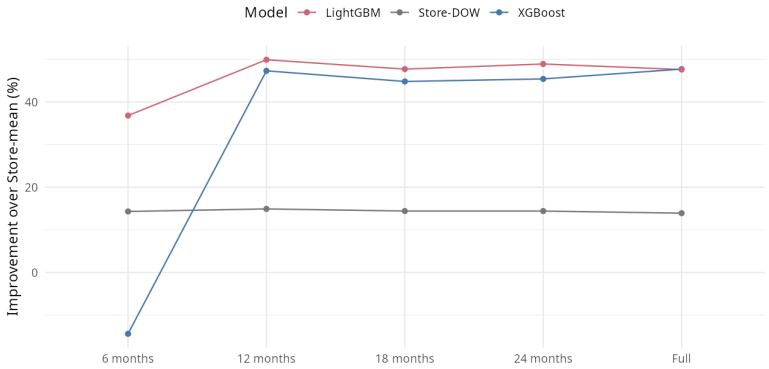


Figure 17: Stage 2 training-window experiment: improvement over the store-mean baseline as the available training history increases.

only 0.1 points at full history. Six months before the test period runs from late December to mid-June, so the training data contain weeks 1 to 25 and the December weeks, but not a single July day. Week enters the model as a number, and a tree can only split it at a value seen in training. Every test week, from 25 to 31, therefore goes to the same branch as week 25 in every tree, so the model treats the whole test period as week 25. That is the last training week. It has only five days, and 100% of its rows are promotion days.

The six-month XGBoost forecasts are too high almost everywhere: the mean forecast is 8354 against actual sales of 6979, and 98% of the stores are overpredicted. Sales did not fall, because Store-mean is too high by only 166, and on the training data the model's average error is close to zero in every month. Splitting each forecast into the part each input adds shows that Week accounts for 840 of the 1026 euros by which the test forecasts exceed those for May and June. The direct test confirms it. When only Week is moved to an earlier training week (20, 23 or 24), the gain rises to between 34.8% and 36.0%, close to LightGBM. The overprediction is also largest on days without a promotion (1950 against 564 on promotion days), which fits a model that forecasts the test period from a week made of promotion days.

A model trained on less than one annual cycle has not seen the calendar positions it is asked to forecast, and a tree model then forecasts them as the nearest position it has seen.

Dimension 3: Store-Type Composition. Table 15 reports results for four store-type subsets.

Table 15: Stage 2 results: store-type composition and baseline strength. Type A comprises 602 stores and Type B 17. The C+D mix comprises 496 stores (148 Type C and 348 Type D); Full includes all 1115 stores across types A, B, C, and D.

Subset	Naïve RMSE	Store-DOW RMSE	Store-DOW Δ (%)	XGBoost Δ_{rel} (%)	LightGBM Δ_{rel} (%)
A only	2004	1674	16.5	47.2	50.9
B only	2638	1822	30.9	62.8	60.9
C+D	1812	1647	9.1	50.1	50.2
Full	1934	1665	13.9	47.7	47.6

Prediction 3 is not supported. Type B has the largest baseline errors and the largest gains for both boosted models. Part of its larger *absolute* RMSE simply comes from Type B's higher sales level, so the percentage measures are the cleaner evidence. Store-DOW improves on Store-mean by 30.9% in this group, and day-of-week means explain about 39% of its within-store variance. Both point to weekly structure as a large part of its baseline error. This holds for the 17 Type B stores in the data; with so few stores it should not be generalised far. The balanced C+D mix, by contrast, gains about as much as Type A (XGBoost 50.1% against 47.2%, LightGBM 50.2% against 50.9%), so mixing two mid-size types leaves the advantage unchanged. Only the distinctive Type B stands apart.

The descriptive evidence in Table 6 explains why the prediction fails. Type B stores are open on almost every Sunday and have average sales well above the other types, so the group labelled most homogeneous by store type is the group with the largest and most distinctive weekly variation. In the terms of the framework, an administrative store label is not a proxy for low $\text{Var}(\delta_{s,t})$: Type B has the *highest* predictable within-store variation of any group. Day-of-week means alone explain about 39% of its within-store variance on average, but the Store-DOW baseline already removes that weekly part, so the boosted models' large gain *over Store-DOW* there comes from the promotion and calendar structure beyond the weekly cycle. This is consistent with Section 4.3.8, where the per-store gain rises with the broader predictable share and falls with the purely weekly share. The lack of support for Prediction 3 is therefore a result about the store-type heuristic used to derive it, not an isolated test of heterogeneity, and it fits Equation (9).

4.3.3 Stage 3: Lag Feature Enrichment

Table 16 compares model performance before and after adding the six lag and rolling-mean features defined in Section 4.2.5.

Table 16: Stage 3 results: Δ_{rel} with and without lag features, full Rossmann dataset. Δ_{DOW} reports improvement over the store-day-of-week baseline.

	Stage 1 (no lags)		Stage 3 (with lags)		
Model	RMSE	Δ_{rel} (%)	RMSE	Δ_{rel} (%)	Δ_{DOW} (%)
LightGBM	1013	+47.6	826	+57.2	+50.2
XGBoost	1003	+48.2	893	+53.7	+46.2
Store-DOW naïve	1665	+13.9	1659	+14.0	–
Store-mean naïve	1934	–	1930	–	–

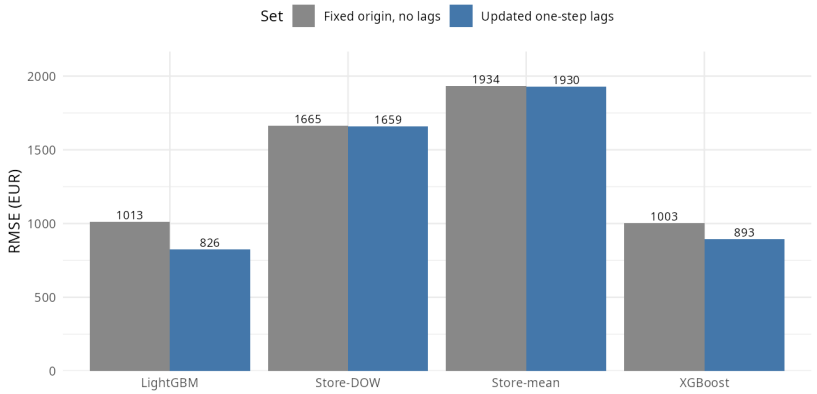


Figure 18: Stage 3 RMSE comparison before and after lag feature enrichment. Adding lag and rolling-mean features substantially reduces the error of both boosted models.

Prediction 4 is supported. Adding lags raises XGBoost's improvement from 39.8% to 46.2% and LightGBM's from 39.2% to 50.2%. The two boosted models are already close without lags and stay within a few percent of each other after enrichment (LightGBM 826, XGBoost 893), with both still far ahead of Store-DOW. These figures use rolling one-day-ahead updating, so part of the lag-feature gain comes from updating the lag inputs with actual sales each day, not from the features alone. The fixed-origin counterpart in Stage 6 (LightGBM 922, XGBoost 980) isolates the smaller, but still clear, contribution of the lags at a frozen forecast origin.

In the fitted models, lag and rolling-mean variables account for about 73% of LightGBM's gain-based feature importance and about 74% of XGBoost's. This share shows where each model puts its splits once the lags are available; it is not separate evidence. Gain-based importance also depends on the exact measure used and is affected by correlated features. With that caveat, it points the same way as the RMSE reduction and supports Prediction 4.

4.3.4 Stage 4: Robustness Across Rolling Test Windows

The final analysis examines whether the Stage 1 ranking is specific to the June–July 2015 holdout period. Table 17 reports the results from four 42-day test windows. Three fall in spring and summer 2015. The fourth covers 30 November 2014 to 10 January 2015, so that the December sales peak, the strongest seasonal pattern in the data, is also tested. All models are re-estimated in each window using only observations dated before the corresponding test period, so this window is trained on data up to late November 2014, having already seen the previous year's peak.

Table 17: Rolling 42-day test-window robustness results (RMSE). Window 4 covers the December peak. Window 1 is the Stage 1 holdout, but the models are refitted separately here, so its values can differ slightly from Table 11.

Window	Test period	Store mean	Store-DOW	XGBoost	LightGBM
1	20 Jun – 31 Jul 2015	1934	1665	1012	1013
2	09 May – 19 Jun 2015	1815	1496	1100	1063
3	28 Mar – 08 May 2015	2287	2044	1451	1367
4	30 Nov 2014–10 Jan 2015	2972	2672	1486	1465



Figure 19: RMSE across four 42-day test windows. Each model is estimated using only observations available before the relevant test period.

Both boosted models lie far below the two baselines in every window, reducing RMSE over Store-mean by roughly 36 to 51%. The advantage is largest in the December window. There the simple baselines are hit hardest by the seasonal peak, and Store-DOW rises to 2672. The boosted models instead capture the peak through the month indicator and the prior year they have already seen, and they improve on Store-DOW by 44 to 45%. Absolute errors rise for every model in this high-volume period, but the relative advantage gets larger. XGBoost and LightGBM stay within 7% of each other in RMSE in every window, so the robust finding is the gap between the boosted models and the baselines, not the order within the pair.

So the result holds over four periods, including the strongest seasonal window. Four windows do not cover every possible forecast origin, but they show that the main ranking does not hang on a single holdout and that the advantage is not an artefact of the spring and summer season.

4.3.5 Stage 5: From a History-Only to a Full Information Set

Table 18 reports the incremental information experiment. Each row pair shows XGBoost and LightGBM re-estimated on one of the four nested feature sets described in Section 4.2.7.

Table 18: Stage 5 results: fixed-origin 42-day forecasts under nested information sets. No test-period sales enter any predictor. Δ_{DOW} is the percentage RMSE improvement over Store-DOW (RMSE 1647 on this split); negative values mean the model is worse than the benchmark.

Information set	Model	RMSE	MAPE (%)	Δ_{DOW} (%)
1. History only	XGBoost	1843	24.3	−11.9
	LightGBM	1852	24.4	−12.4
2. + Calendar	XGBoost	1356	17.4	+17.7
	LightGBM	1306	16.4	+20.7
3. + Store attributes	XGBoost	1427	18.6	+13.3
	LightGBM	1286	16.1	+21.9
4. + Promotions and holidays	XGBoost	1223	15.0	+25.8
	LightGBM	1182	14.3	+28.2



Figure 20: Stage 5 information path: test RMSE of XGBoost and LightGBM across the four nested information sets, with the Store-DOW reference level reconstructed from the reported percentage gains (rounding applies).

Figure 20 traces these RMSE values across the four nested sets. The history-only condition is the most revealing. With nothing but store identity and origin-safe summaries of its own sales, XGBoost and LightGBM are 11.9% and 12.4% *worse*

than Store-DOW (RMSE 1843 and 1852 against 1647) and far behind the store-level Auto-ARIMA fits (1508). In a univariate information environment, a well-specified statistical model fitted to each series is clearly the better choice. This echoes, inside a single dataset, the pattern observed in the early M competitions, and it suggests that the weak machine-learning results of that period reflected more than algorithm quality, since even current implementations of XGBoost and LightGBM can do little with this information set.

Adding calendar variables changes the picture sharply. Both models drop well below Auto-ARIMA (XGBoost 1356, LightGBM 1306), and LightGBM gains more from the calendar information than XGBoost. Store attributes do not help and set XGBoost back, raising its RMSE from 1356 to 1427, while LightGBM improves only marginally (1306 to 1286). This is what one would expect once the store identifier has already absorbed the stable differences between stores. Type, assortment, and competitor variables then have little within-store predictive content left to add. Why the two models react differently, for example through capacity or feature selection, is not something this experiment can isolate. Promotion and holiday indicators then improve both fits, to 1223 and 1182. Calendar variables give the largest single drop for both models, although this depends partly on the order in which the feature groups are added.

This progression supports Prediction 6. With only its own sales history each boosted model does no better than the simple and statistical baselines, and the ensemble advantage grows as calendar, store, and promotion information is added.

Testing the reversal formally. The claim above, trees behind Auto-ARIMA with history only, trees ahead with the full feature set, is so far only a comparison of RMSE point estimates. The same Diebold–Mariano procedure used in Section 4.3.7 (Equation (31)) is therefore applied to both regimes. Here $e_t^{(1)}$ is the boosted model's daily loss and $e_t^{(2)}$ is Auto-ARIMA's, so a negative DM statistic means the boosted model has the lower loss and a positive one means Auto-ARIMA does:

$$H_0 : \mathbb{E}\left[e_t^{(1)} - e_t^{(2)}\right] = 0, \qquad H_1 : \mathbb{E}\left[e_t^{(1)} - e_t^{(2)}\right] \neq 0. \tag{30}$$

The two feature regimes do not use exactly the same stores and dates, because each is trimmed to its own warm-up needs: the origin-safe lags of Stage 5 need 84 trading observations of earlier history per store, and Stage 1 needs none. The two sets of tree predictions and the Auto-ARIMA forecasts are therefore inner-joined on (store, date) before testing, so every test in Table 19 compares the three models on the same rows within its regime. The tests are run in `rossmann_stage7_tree_vs_statistical.Rmd`, a separate file, because the Stage 5 pipeline does not keep row-level predictions and never fits Auto-ARIMA, so no earlier file has the aligned forecasts that a paired test needs.

Table 19: Diebold–Mariano tests (Equation (30)) for the tree-versus-statistical reversal, full-feature (Stage 1) and history-only (Stage 5, condition 1) regimes. Negative DM favours the boosted model. Positive favours Auto-ARIMA. p-values Holm-adjusted across all four tests.

Comparison	DM	Holm p	Favours
Full-feature: XGBoost vs Auto-ARIMA	-4.58	< 0.001	XGBoost
Full-feature: LightGBM vs Auto-ARIMA	-3.87	0.001	LightGBM
History-only: XGBoost vs Auto-ARIMA	3.68	0.001	Auto-ARIMA
History-only: LightGBM vs Auto-ARIMA	3.73	0.001	Auto-ARIMA

All four tests have Holm-adjusted $p < 0.01$ on the aligned rows. The same file also recomputes the RMSEs: about 1003 and 1013 for full-feature XGBoost and LightGBM, and 1840 and 1852 for their history-only fits, against 1508 for ARIMA. The history-only XGBoost value differs from Stage 5's 1843 because these are separate fits. The reversal holds, but the tests use `h=1`, so their variance estimate ignores dependence between neighbouring days. They should be read as nominal evidence for this setting, not as a final significance result for the model families.

The best fixed-origin RMSE here (1182) is higher than the Stage 1 no-lag result (1013): the origin-safe history features, whose shortest lag is 42 trading observations, carry less signal than the calendar and promotion variables already present in Stage 1, and the warm-up rows dropped to construct them shrink the training sample. The rolling Stage 3 result (826) remains far better than any fixed-origin specification, so a large part of the Stage 3 advantage reflects daily updating rather than information available at the origin. The sequential attribution in Table 18 also depends on the order in which feature groups are added. The order used here follows the historical development from univariate to covariate-rich environments, and single-condition results should be read with that in mind.

4.3.6 Covariate-Matched ARIMAX Robustness Check

Up to this point the statistical models see only each store's past sales, while the trees also see the calendar and the promotion plan. Part of the tree advantage could therefore come from this extra information rather than from the trees themselves. This check removes that difference. Regression with ARIMA errors (ARIMAX), XGBoost and LightGBM all receive the same covariates: weekday indicators, promotion and its interaction with the weekday, school and state holidays, whether the store is open, an annual sine and cosine pair and a linear time trend. ARIMAX is fitted separately for every store, so each store gets its own covariate effects. The trees are fitted once on all stores together and also receive store indicators. Opening days and the promotion plan are treated as known in advance, which is how the Kaggle test file provides them.

The store sample is fixed before any model is fitted. Only stores with a complete daily record up to the first origin can be used, and 50 of them are drawn with seed 230923, stratified by type as 27 A, 1 B, 7 C and 15 D. Forecasts start on 8 May and on 19 June 2015 and run for 42 days. Every model trains on the same daily rows, closed-day zeros included, and every model is scored on the same open days with positive sales. No test sales are used. Auto-ARIMA without covariates is fitted on the same stores, so the effect of the covariates alone can be read inside this experiment. Because the experiment uses only 50 stores, its RMSE values should not be compared one to one with the full-panel results earlier in this section.

ARIMA orders are chosen by AICc with a stepwise search, with $p, q \leq 3$, $P, Q \leq 1$ and a combined order of at most 5. Covariate columns that are constant or duplicated in the training window are dropped, using training data only. For the trees, the number of rounds is chosen on the last 42 training days, and the model is then refitted on the whole training window with that number. Tree size is fixed at depth 6 for XGBoost and 31 leaves for LightGBM. The information is matched in this way, but not everything else: the model families, their loss functions and the amount of tuning still differ, and ARIMAX is still local while the trees are pooled.

Table 20: Covariate-matched comparison: 50 stores, two fixed origins. All models at an origin are scored on the same store–day rows.

Origin	Model	RMSE (EUR)	MAE (EUR)	Test rows
2015-05-08	ARIMAX	887	652	1684
2015-05-08	LightGBM	901	676	1684
2015-05-08	XGBoost	1018	723	1684
2015-05-08	Store-DOW	1446	1095	1684
2015-05-08	Auto-ARIMA	2098	1621	1684
2015-06-19	LightGBM	754	561	1812
2015-06-19	XGBoost	822	607	1812
2015-06-19	ARIMAX	894	652	1812
2015-06-19	Auto-ARIMA	1361	1051	1812
2015-06-19	Store-DOW	1572	1216	1812

Adding the covariates helps ARIMAX a lot: it beats Auto-ARIMA by a wide margin at both origins. At the May origin its RMSE is 887, against 901 for LightGBM and 1018 for XGBoost. The paired interval for LightGBM against ARIMAX includes zero, and the one for XGBoost against ARIMAX excludes zero in favour of ARIMAX. At the June origin the picture turns around. ARIMAX reaches 894, LightGBM 754 and XGBoost 822; the interval for LightGBM excludes zero in favour of LightGBM and the one for XGBoost excludes zero in favour of XGBoost. The interval for Auto-ARIMA against ARIMAX excludes zero in favour of ARIMAX at both origins. So once the statistical model gets the same information, a large part of the gap disappears, and which model wins depends on the forecast period.

Table 21: Paired loss differences with 95% circular block-bootstrap intervals. Each difference is the comparator's MSE minus the ARIMAX MSE, in 10^3 EUR^2, so positive values favour ARIMAX. The bootstrap resamples whole days in seven-day blocks, 2000 times, which keeps the dependence between nearby days and between stores on the same day.

Origin	Comparator	Difference	Lower	Upper
2015-05-08	Auto-ARIMA	3617.6	2009.4	5349.3
2015-05-08	XGBoost	250.7	129.7	376.1
2015-05-08	LightGBM	26.3	−101.1	133.1
2015-06-19	Auto-ARIMA	1053.5	621.2	1478.3
2015-06-19	XGBoost	−123.4	−257.2	−12.3
2015-06-19	LightGBM	−230.9	−361.6	−103.7

Statistical models do not simply lose once they are given useful covariates.

These intervals hold for this store sample and these two origins only. A 42-day holdout contains just six seven-day blocks, so the sampling uncertainty is large. Type B has a single store in the sample, so nothing can be said about that type. The question here is whether ARIMAX closes the gap to the trees, not which family has to win. Answering it more generally would need the same design on more origins and on other datasets.

Shorter training histories. One difference between the two families is left. The trees learn from all 50 stores at once, while each ARIMAX model sees only its own store. Learning across stores should help most when each store has little history of its own. To test this, the same design is run again with only 3, 6 or 12 months of training data before the same origins (`rossmann_arimax_history.Rmd`). Everything else stays the same, with one exception: the annual sine and cosine terms are dropped from all models when the window is shorter than a year, because with less than one yearly cycle they are poorly determined and easily confused with the trend. The covariate set is therefore not identical across window lengths. Table 22 shows the results.

A shorter history does not make pooling win more often. At the May origin, none of the four intervals favour LightGBM and none favour ARIMAX. At the June origin, four favour LightGBM and none favour ARIMAX. Over all eight combinations of origin and window, LightGBM is significantly better in four and ARIMAX in none. On average the RMSE is 967 for LightGBM and 1098 for ARIMAX. ARIMAX also moves around more: its RMSE lies between 887 and 1422, against 754 and 1200 for LightGBM. All windows use the same two test periods, so this is really two independent comparisons and not eight. What decides the order is the forecast period, not the length of the training history.

Table 22: ARIMAX and pooled trees under shorter training histories: RMSE (EUR) on the same 50 stores and test rows. The last three columns give LightGBM MSE minus ARIMAX MSE in 10^3 EUR^2 and the bounds of its 95% circular block-bootstrap interval; negative values favour LightGBM. Full-history rows repeat Table 20.

		RMSE (EUR)				LightGBM − ARIMAX		
Origin	History	ARIMAX	LightGBM	XGBoost	Store-DOW	Diff.	Lower	Upper
2015-05-08	3 months	1056	1200	1109	1403	325	−15	672
	6 months	1095	1091	1070	1377	−8	−490	449
	12 months	974	1007	1121	1369	64	−67	186
	Full history	887	901	1018	1446	26	−101	133
2015-06-19	3 months	1307	1017	1036	1569	−672	−1217	−48
	6 months	1422	925	929	1545	−1166	−1858	−488
	12 months	1147	840	915	1543	−610	−894	−296
	Full history	894	754	822	1572	−231	−362	−104

Why the tree models remain the main models. This result supports the main argument of the thesis more than it ranks the two families. With the same covariates, the same statistical model gets much better: ARIMAX lowers the Auto-ARIMA RMSE by 58% at the May origin and 34% at the June origin. Stage 5 shows the same thing for the trees when calendar, promotion and store information is added. The conclusion therefore does not depend on the method: most of the gap between the univariate benchmarks and the trees comes from the information the models get, not from the model class.

ARIMAX is still not used in the other experiments, for two reasons. First, a per-store ARIMAX forecast does not change when other stores are added or removed, because each model sees only its own store. The scale, heterogeneity and global-versus-local experiments of Stage 2 and Section 4.3.7 are about exactly that, so ARIMAX could only serve there as a fixed reference. Second, the ARIMAX pipeline used here requires a complete daily record. Only 934 of the 1115 stores have one up to the first origin, while the trees can use every store as it is. Even with the same information, ARIMAX is not significantly better than LightGBM in any combination of origin and window, and its errors change more from one forecast period to the next.

4.3.7 Stage 6: Additional Analyses

Six further analyses check how solid the main results are. They are run in a separate file (`rossmann_stage6.Rmd`) with the same pipeline as the other stages. The recursive-lag analysis below compares against the Stage 1 no-lag values (XGBoost 1003, LightGBM 1013). The hyperparameter grid is a separate run, so its settings closest to Stage 1 do not reproduce those values exactly.

Fixed-Origin Recursive Lag Forecasts. Stage 3 changes two things at once: the lag features add information, and the rolling scheme adds daily updating. To separate them, the lag models are run again with recursive forecasts. During the 42-day holdout the lag and rolling features are computed from the model's own predictions, so nothing after the forecast origin is used.

Estimated this way, LightGBM reaches an RMSE of 922 and XGBoost 980, compared with 1013 and 1003 without lags. Lag information available at the origin therefore accounts for an improvement of about 9% for LightGBM but only about 2% for XGBoost, with the remaining distance to the rolling Stage 3 results (826 and 893) attributable to daily updating. The two components therefore matter very differently across the pair: for LightGBM the origin-available lag information and the daily updating contribute about equally, whereas XGBoost gains almost everything from updating and little from the lags available at the origin. Either way, the Stage 3 gains are not an artefact of the rolling scheme. A substantial part of them survives at a fixed origin.

Statistical Significance of the Main Comparisons. Every RMSE comparison so far compares two point estimates. This paragraph adds a formal test for the two comparisons the results rely on most: the boosted trees against Store-DOW, and XGBoost against LightGBM. Both use the same Diebold–Mariano procedure, so the two hypotheses are tested in the same way. The tests run on the daily mean squared errors of the primary holdout, 42 daily values per model. For two forecasts with squared-error loss series $e_t^{(1)}$ and $e_t^{(2)}$, the daily loss differential is $d_t = e_t^{(1)} - e_t^{(2)}$. The hypotheses have the form already stated in Equation (30), $H_0 : \mathbb{E}[d_t] = 0$ against $H_1 : \mathbb{E}[d_t] \neq 0$, and are tested with

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{\widehat{\mathrm{Var}}(\bar{d})}}, \qquad \bar{d} = \frac{1}{T}\sum_{t=1}^{T} d_t, \tag{31}$$

where $T = 42$ is the number of daily losses. The archived scripts call `forecast::dm.test` with `h=1`. With this setting the function uses a lag-zero variance estimate, a small-sample correction and a t_{41} reference distribution. It does not correct for serial correlation the way a HAC estimator would. Daily losses from the same fixed origin can be correlated, so the nominal significance levels should be read with care.

The archived statistics are -4.67 and -5.11 for LightGBM and XGBoost against Store-DOW, and 2.44 for LightGBM against XGBoost. They come from this original test setup and do not show that the result is robust to dependence over time. The last comparison has nominal $p = 0.02$ but only a small RMSE difference. The ARIMAX experiment instead uses paired seven-day block-bootstrap intervals, and its short 42-day span is stated openly.

A test for the research question itself. The six predictions in Section 3.7 turn the research question into testable statements, but the question can also be put to one direct test. Across different stores, rather than across experimental conditions built from the same pooled sample, does the boosted-tree advantage over StoreDOW move with the predictable within-store variation $\mathrm{Var}(\delta_{s,t})$, as the framework says it should? Section 4.3.8 answers this with a per-store regression of each store's RMSE gain on measurable data properties, among them the weekly-predictable share of variance and the broader predictable share. Each coefficient is tested against $H_0 : \beta = 0$, meaning the property is unrelated to the gain, with $H_1 : \beta \neq 0$. The joint hypothesis $H_0 : \beta_1 = \cdots = \beta_8 = 0$ is tested with the omnibus F-test reported there. Table 24 gives the coefficients and t-statistics; $|t| > 1.96$ rejects the per-coefficient H_0 at the 5% level, and $R^2 = 0.26$ is the share of the cross-store variation in gain that the regression explains. This is the test that targets the research question most directly, because it runs over 1115 individual stores instead of the handful of aggregate conditions used elsewhere in Section 4.

Replication uncertainty in the number-of-stores experiment. The values in parentheses in Table 13 are standard errors of the mean over the 20 draws, not standard deviations. For LightGBM they are 0.87, 0.76 and 0.60 percentage points at 50, 200 and 500 stores; multiplying by $\sqrt{20}$ gives the spread across draws. The full panel is fitted once.

Hyperparameter Sensitivity of the LightGBM–XGBoost Ranking. The Stage 1 configurations differ in capacity, with 63 leaf-wise leaves against depth-6 trees. A grid over XGBoost depth $\{6, 8, 10\}$ and LightGBM leaves $\{31, 63, 127\}$, with the learning rate and early-stopping procedure held fixed, tests whether the ranking between the two models survives other capacity choices. Table 23 reports the no-lag results on the primary holdout.

Table 23: Hyperparameter grid on the primary holdout, no lag features, ordered by RMSE. These are the archived Stage 6 results, a separate run from Stage 1. Differences between repeated fits are not estimates of run-to-run variance, because no controlled replication was run.

Model	Capacity setting	RMSE
XGBoost	max depth = 8	948
XGBoost	max depth = 10	962
XGBoost	max depth = 6	970
LightGBM	127 leaves	988
LightGBM	63 leaves	1013
LightGBM	31 leaves	1068

The grid favours XGBoost across its three tested settings. The best result is an XGBoost model at a max depth of 8 (948), and all three of the XGBoost settings overtake the widest LightGBM configuration, although capacity helps only up to a point, since depth-8 edges out depth-10. Two conclusions follow. First, the XGBoost–LightGBM ordering reported in Stage 1 should be read as a comparison of common configurations, not of algorithms. Capacity changes the margin, but this grid does not establish a reversal of the Stage 1 XGBoost–LightGBM ordering. Second, the joint advantage of the boosted ensembles over the baselines is robust across every cell of the grid, since even the weakest configuration (1068) beats Store-DOW.

A Direct Test of the Framework. Equation (9) predicts that the ensemble gain should increase with the amount of predictable within-store variation, and this is testable directly at the store level. For each store, the predictable-variation proxy is the variance of store-specific training day-of-week and promotion cell means (in squared sales units), and the realised gain is the store's reduction in test MSE from Store-DOW to LightGBM.

The cross-store correlation between the proxy and the realised MSE reduction is 0.46 ($p < 0.001$, $n = 1115$), so stores with more predictable variation do gain more from the ensemble, as the framework predicts. The relationship is still far from deterministic, since the proxy is coarse and explains only about 21% of the cross-store variance in gains. Its sign and size fit the proposed mechanism, but they do not prove it causally, and store size could still be partly behind it.

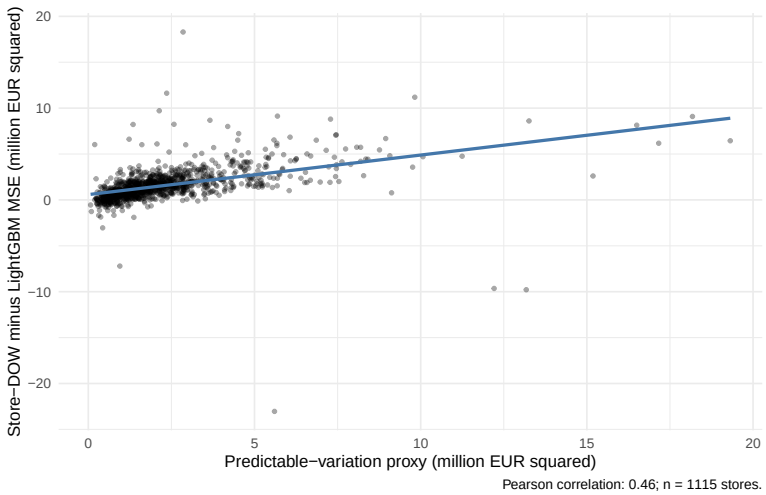


Figure 21: Store-level test of the framework: predictable within-store training variance against the realised MSE reduction of LightGBM relative to Store-DOW on the holdout.

Scale with Fixed Composition and a Global–Local Comparison. Two final checks return to the rejected scale prediction. First, the number-of-stores experiment

is repeated with samples drawn only from Type A stores, so that composition is held fixed while scale changes, extended up to the full set of 602 Type A stores. Under this design LightGBM's gain over Store-mean rises overall and then approximately flattens, at 45.0%, 50.2%, 51.5%, and 50.9% for 50, 200, 400, and the full 602-store set (standard errors 5.3, 2.7, and 1.3 over five draws for the sampled conditions). XGBoost moves the same way, from 44.9% at 50 stores to between 48.1% and 50.1% at the larger sizes, rather than the decline seen in the mixed samples. Much of the downward pattern in Table 13 therefore reflects changing store composition rather than the number of series itself.

Second, a global LightGBM model is compared with separate per-store LightGBM models on a common sample of 200 stores, using the no-lag features in both cases. The local models reach an RMSE of 835 against 935 for the global model restricted to the same stores. On this sample, fitting each store separately is clearly better than pooling. This is consistent with the scale results and suggests that learning across stores is not what drives the boosted models' advantage in Rossmann, where each store already has around 760 training observations of its own. The comparison uses one fairly conservative local setup and one global setup, so it is indicative, not complete, but its direction matches the conclusion of Montero-Manso and Hyndman (2021) that the value of pooling depends on the data environment.

4.3.8 Data Properties and Generalisability

The experiments above vary operational settings: the number of stores, the training length, the store-type composition, and the feature set, but any such setting is by definition specific to Rossmann. To state a result that could carry to another retail dataset, the advantage is therefore re-expressed in terms of *measurable data properties* of each store's series, the quantities that Equation (9) says should matter.

A finding such as "more stores did not help" is not meant to transfer mechanically to another retailer. What carries over to other data is the diagnosis: whether additional stores increase shared learnable structure, whether the dominant variation is already captured by a seasonal baseline, and whether the feature set makes the remaining predictable movements observable.

For every store, a small set of series-level properties is computed on the training data: the coefficient of variation `cv`, the share of within-store variance explained by store-specific day-of-week means (`dow_strength`), the share explained by day-of-week and promotion jointly (`pred_share`, a proxy for $\mathrm{Var}(\delta)/\mathrm{Var}(y)$), a linear trend strength, the promotion frequency, the closed-day fraction, the mean sales level, and the series length. The realised gain of one global LightGBM over the Store-DOW baseline on the 42-day holdout is then regressed on these standardised properties. This is implemented in `rossmann_data_properties.Rmd`. Across the 1115 stores the gain over Store-DOW has a median of 28.8% and a mean of 16.9% (the mean is pulled down by a thin tail of stores the global model fits poorly). The regression explains about a quarter of the cross-store spread in this gain (R^2 =

0.26, $F_{8,1106} = 48.9$). Table 24 reports the standardised coefficients. They are clearly signed and jointly significant, but the modest R^2 matters for interpretation, because the remaining three-quarters of the cross-store variation reflects estimation noise from the single 42-day window, store-specific demand structure that the coarse property set does not capture, and non-linear interactions a linear regression omits. The property mapping should therefore be read as a tendency that orders stores on average, not as a precise per-store predictor of the gain.

Table 24: Per-store gain over Store-DOW (%) regressed on standardised series-level properties (1115 stores). Coefficients are comparable in size because every predictor is standardised.

Property	Coefficient	t
`pred_share` (Var(δ) proxy)	+14.74	8.7
`mean_sales`	+14.58	13.1
`cv` (variability)	+9.96	7.4
`trend_strength`	+4.61	3.8
`length`	−0.13	−0.1
`promo_frac`	−0.38	−0.1
`closed_frac`	−2.42	−0.8
`dow_strength` (weekly share)	−19.87	−13.0

The pattern refines the framework. The predictable share `pred_share`, the proxy for Var(δ), carries a large positive coefficient, so stores with more predictable within-series variation gain more from the tree, consistent with Equation (9). The coefficient on `dow_strength` is large and *negative*. This is expected: the benchmark here is Store-DOW, which already captures pure day-of-week structure, so a store whose predictability is mostly weekly leaves little for the tree to add. The gain therefore comes from predictable variation *beyond* the weekly cycle, the promotion and calendar interactions that `pred_share` picks up but a day-of-week mean cannot. Variability (`cv`) and a faint trend also widen the gap, while the promotion frequency, closed-day fraction, and series length are insignificant once these are controlled for. The one term not in Equation (9) is the sales level (`mean_sales`), which enters positively. Larger stores realise larger percentage gains even after the coefficient of variation is held fixed, so the effect is more than high-volume stores being less noisy in relative terms. A plausible channel is the global objective itself: the pooled model minimises squared error in euros, so high-volume stores carry more weight in the loss and the model fits their recoverable component more precisely. That raises their realised R_{rec} and so their gain, which means the level acts through the same recovery term already in the framework rather than against it. It remains the one result the decomposition does not predict directly, and isolating it on a dataset with a different sales scale would be a useful check.

Because these properties can be measured on any retail series, the mapping from properties to expected gain, rather than the Rossmann-specific RMSE numbers, is the part of the finding that would transfer to a different dataset. A series with high predictable share beyond its weekly pattern and high variability is where a tree-based model is expected to pay off. One whose structure is almost entirely weekly, and so already captured by a seasonal baseline, is not.

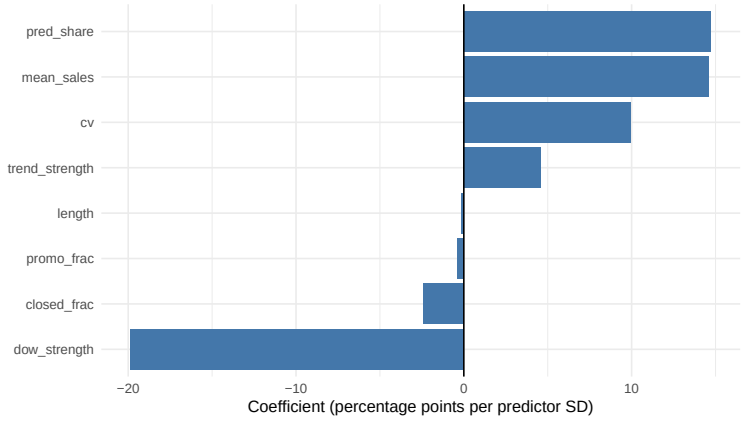


Figure 22: Which data properties predict the tree advantage. Standardised coefficients from a regression of the per-store percentage gain over Store-DOW on the series-level properties.

4.3.9 Direct Answer to the Research Question

Table 25 brings the separate experiments together. The reported values measure the reduction in RMSE relative to Store-DOW, the stronger of the two simple store-level baselines. Positive values therefore mean that the tree-based ensemble outperforms a forecast that already accounts for store identity and weekly seasonality. The table also includes the conditions where the boosted models lose, the history-only feature set and, for XGBoost, the six-month window, so the advantage is shown where it reverses as well as where it holds. With only a store's own sales history the boosted models trail Store-DOW by about 12%, and the advantage appears only once calendar, store, and promotion information is added, growing further with sales lags. This fits the story of the M competitions, but it is an illustration, not a proof of why the competitions differed.

Table 25: Percentage reduction in RMSE relative to Store-DOW across the experimental conditions. Positive values mean the boosted trees beat the baseline. Negative values mark the settings where they fall behind it. The history-only and calendar, store and promotion rows both come from Stage 5 (Table 18), so they use the same rows and the same origin-safe history features. The lag-enriched row comes from Stage 3.

Dimension	Condition	XGBoost	LightGBM
Number of stores	50 stores	+45.9	+45.7
	200 stores	+43.0	+44.4
	500 stores	+41.2	+43.7
	1115 stores	+39.2	+39.2
Training history	6 months	−33.5	+26.3
	12 months	+38.1	+41.1
	18 months	+35.5	+38.9
	24 months	+36.2	+40.3
	Full history	+39.3	+39.1
Store composition	Type A	+36.8	+41.3
	Type B	+46.2	+43.4
	C+D mix	+45.1	+45.2
	Full sample	+39.2	+39.2
Feature information	History only	−11.9	−12.4
	+ calendar, store, promotion	+25.8	+28.2
	+ lags and rolling means	+46.2	+50.2
Test period	Window 1	+39.2	+39.2
	Window 2	+26.5	+28.9
	Window 3	+29.0	+33.1
	Window 4 (Dec.)	+44.4	+45.2

Both XGBoost and LightGBM outperform Store-DOW in every reported condition except two: the history-only feature set, where both trail it by about 12%, and the six-month training window for XGBoost, the case explained in Section 4.3.2. With six months LightGBM still beats Store-DOW, and both models become clearly stronger once at least 12 months of history are available. The largest full-sample gains come after recent sales lags and rolling means are added, lifting XGBoost to 46.2% over Store-DOW and LightGBM to 50.2%, while the number-of-stores experiment moves in the opposite direction to the original scale prediction, so raw cross-sectional size is not the condition that explains the advantage.

So the answer depends on data properties, not on the experimental levers: what decides the gain is whether the series hold predictable variation the baseline leaves unexplained and whether the feature set makes it observable. Section 5 develops that reading against the six predictions.

4.3.10 What the Experiments Establish

Table 26 separates what the experiments show about information access from what they cannot show about hidden demand components. A lower loss shows that a model used more information well; it does not identify which unobserved part of demand it recovered.

Table 26: Information access and interpretation limits.

Comparison	Evidence	Interpretation limit
Store-mean vs Store-DOW	Weekly conditioning lowers error.	Not a measure of all predictable demand.
History-only trees vs ARIMA	ARIMA wins in archived results.	History representation and recency differ.
Nested tree features	Calendar and commercial features improve fitted models.	No causal effect or universal family ranking.
Updated lag features	Recent observations improve one-step forecasts.	Different information from fixed-origin forecasting.
Matched covariates	Section 4.3.6 compares ARIMAX and trees.	Two origins and 50 eligible stores limit generalisation.

5 Conclusion

Table 27 summarises the outcome of the six predictions before the detailed discussion.

Table 27: Summary of the framework's predictions and the empirical outcomes.

#	Prediction	Outcome	Key evidence
1	Gain increases with the number of stores	Rejected	Relative gain falls significantly as stores are added (Table 13)
2	Gain increases with training length	Partially supported	Large gain from 6 to 12 months; flat thereafter (Table 14)
3	Homogeneous store groups show smaller gains	Not supported	Type B has the largest gains and the largest average weekly share; the latter does not by itself explain the incremental gain (Tables 15, 6)
4	Lag features improve the boosted models	Supported	RMSE falls 1003→893 (XGBoost) and 1013→826 (LightGBM) (Table 16)
5	Regularised ensembles outperform simpler trees	Partially supported	Boosted trees take the top two, but un-regularised bagging is third and beats the random forest (Table 11)
6	Advantage grows as the information environment is enriched	Supported	History only trails Store-DOW and Auto-ARIMA; full features overtake both (Table 18)

This thesis asked which measurable properties of retail demand data explain when tree-based ensembles beat simple baselines. Rossmann gives a clear answer: the advantage comes from predictable variation that the baseline leaves unexplained and that the feature set makes visible. The baseline comparison is the first part of that answer. Store-DOW cuts RMSE by 13.9% against Store-mean, so a large part of Rossmann demand can already be forecast from store identity and the weekly pattern. Even against this harder benchmark, XGBoost and LightGBM cut RMSE by about 39% without lag features on the primary Stage 1 holdout; the exact margin differs across the other windows and separately executed runs. Their success is therefore not an artefact of a weak store-mean benchmark. It comes from predictable

structure beyond the weekly pattern. When the models are given only a store's own sales history, they fall behind both Store-DOW and Auto-ARIMA, which shows that a flexible algorithm alone is not enough.

The ARIMAX check puts a limit on this conclusion. With the same covariates, ARIMAX keeps up with LightGBM at one origin and loses at the other, and shorter training histories do not change this: it is not significantly better than LightGBM in any combination of origin and window. The conclusion is therefore conditional. Trees can use structure beyond a seasonal baseline, but whether they beat statistical forecasting can only be judged when both get the same covariates and the same forecast origins. The univariate comparison on its own cannot show that.

5.1 Reconciling the Rejected and Unsupported Predictions

Two of the six predictions fail: Prediction 1 (scale) is rejected, and Prediction 3 (heterogeneity) is not supported. Both were stated in terms of operational levers, the number of stores and an administrative store-type label, and in both cases the lever turns out to be a poor proxy for the quantity in Equation (9) it was meant to track. The decomposition itself remains an organising identity, not a causal mechanism that has been independently confirmed. These two results are a real challenge to the initial hypotheses. Store composition is a plausible explanation for them, not proof that the mechanism survives every test, and the property regressions are an exploratory refinement that still needs independent validation.

The experiments on scale, training length, and store type clarify what the relevant data properties are not. More cross-sectional size does not automatically lift a global tree model's relative performance, since both boosted models gain more in the smaller store subsets than in the full mixed sample (Table 13). The fixed-composition and global–local checks in Section 4.3.7 sharpen this point, since LightGBM's gain stops falling when composition is held fixed and per-store models beat the pooled model on a common sample. Training length also matters only up to a point: results are weakest at six months, jump at twelve, and then flatten (Table 14), suggesting that Rossmann needs one full annual cycle but gains little from older observations beyond that. The store-type experiment points in the same direction, since the supposedly homogeneous Type B subset yields the largest gains rather than the smallest, despite its strong weekly baseline. It also has the largest average weekly share in Table 6, but that alone does not explain why the trees gain so much there. So neither the number of series, the length of the history nor an administrative label is the explanation that carries over. Each matters only through the predictable structure and baseline strength it brings with it.

Feature information is the strongest and most stable driver. Adding lags and rolling means drops both boosted models far below the baselines (Table 16), and these variables account for most fitted-model importance once they are available. The incremental information experiment ties this result back to the literature review:

with only a few old summaries of their own history, XGBoost and LightGBM lose clearly to per-store Auto-ARIMA. Once calendar variables are added they beat the statistical benchmarks clearly, and the calendar indicators give their largest single gain. The path of the M competitions, where statistical methods held their ground until tasks with rich, aligned covariates appeared, therefore repeats inside one retail dataset when the algorithm and sample are fixed and only the information set changes. This is the central empirical thread of the thesis: tree ensembles win when the feature set exposes recent and recurring demand movements, not whenever the dataset is larger or the algorithm more flexible.

Between the two boosted implementations there is no stable winner: their RMSE difference changes across separately executed fits and test windows. The grid cannot pick a best model either, because every cell is scored on the same holdout, so choosing the winner there would build in selection bias. Rossmann thus reproduces the broad M5 finding that boosted trees work very well for tabular retail forecasting, while showing that the gap between the two implementations depends on the chosen configurations rather than on the algorithms.

The evidence, taken together, gives no support to a tidy rule that tree-based methods win because a dataset is larger, longer, or more heterogeneous. In Rossmann the deciding condition is whether the model sees variables that capture recent and recurring demand patterns. A long enough history and a meaningful cross-section are needed for estimation, but past those minimums, feature relevance counts for more than raw dataset size. The Stage 1 comparison reaches the same verdict against dedicated statistical models on the primary holdout, where LightGBM and XGBoost beat Auto-ARIMA, Theta, and ETS and Random Forest finishes clearly ahead of Auto-ARIMA. The broader result is strongest for the two boosted ensembles, not for tree-based methods as an undifferentiated group.

5.2 Practical Implications

For a practitioner with a few hundred to a few thousand store-level series and standard calendar and promotion information, several things follow. Compute a store and day-of-week average before crediting any complex model, since it removes about 14% of the store-mean error at almost no cost and makes weak models easy to spot. Feature work then pays off before model work, since sales lags and rolling means produced larger gains than any change in dataset size and about three quarters of the fitted models' importance lands on them. Roughly one full year of history is enough in this setting, and gathering more stores or longer histories will not raise relative performance by itself. Finally, check model advantages over several forecast origins, because their size, but not their direction, drifts across test windows.

5.3 Limitations and Future Work

The conclusions hold for the designs that were fitted. The ARIMAX comparisons deal with one important information confound, but they cover only 50 eligible stores and two origins. The full-panel tables are read from the executed reports of the individual analysis scripts, which were run separately rather than together in one computing environment. Integer coding of categories, imputation before the split and the different ways the history is represented remain limits of those archived comparisons.

There are further limitations. The evidence comes from one dataset of aggregate store sales, so the same framework should be run on product-level data such as a subset of M5, where demand is more intermittent. Stage 3 uses rolling one-day-ahead updates, so its lag gains are not fixed-origin 42-day results; the recursive analysis in Section 4.3.7 addresses this. The LightGBM–XGBoost comparison sets two fitted specifications against each other under one tuning budget, and the grid in Section 4.3.7 shows that their order depends on capacity, even though their joint advantage over the baselines does not. The Type B result rests on only 17 stores. Every forecast here is a point forecast, so the probabilistic evaluation that was central to the M5 Uncertainty track is left out. Finally, most comparisons rest on a single 42-day holdout, so the headline RMSE rankings are point estimates without formal standard errors. The rolling windows of Stage 4 and the cross-store regression in Section 4.3.8 protect the main claims against a single-window artefact, but a fuller treatment would use many more rolling origins and attach uncertainty to every reported number. Estimating the store-level link between predictable variation and realised gain (Section 4.3.7) on several datasets would test whether recent demand features stay central in more intermittent and more detailed retail settings.

Reproducibility

The source is `Kayra_Isik_thesis_final_v3.Rnw`. Every table and summary plot is drawn from machine-readable result files. The audit file `table_provenance.csv` records the executed report or CSV behind each archived result. These archived results are not presented as one new, synchronised rerun of the full panel, and differences between repeated specifications are kept with their source. The covariate-matched comparison is reproduced by `rossmann_arimax.Rmd` and the training-history check by `rossmann_arimax_history.Rmd`; their samples, predictions, model orders, uncertainty calculations and session details are kept. The six-month diagnosis in Section 4.3.2 is reproduced by `rossmann_window6_diagnostic.Rmd` from the saved six-month fit. A failed fit cannot silently drop a store, and a missing result table stops compilation. Earlier versions of the code are available at https://github.com/kayraisik/Rossmann-M5_thesis.

Use of Generative AI

Generative AI assisted explanation, drafting, code development and review. The author directed the research question and original design. AI assistance also proposed methodological corrections and helped implement this revision's covariate-matched analysis and its training-history follow-up. Numerical outputs are linked to executed code or documented archived reports. Computational checks do not replace the author's review of assumptions, interpretation and citations. The author is responsible for understanding the methods and reviewing and accepting responsibility for the submitted text and analysis.

References

Assimakopoulos, V. and Nikolopoulos, K. (2000). The theta model: a decomposition approach to forecasting. *International Journal of Forecasting*, 16(4):521–530.

Bojer, C. S. and Meldgaard, J. P. (2021). Kaggle forecasting competitions: An overlooked learning opportunity. *International Journal of Forecasting*, 37(2):587–603.

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Chapman and Hall/CRC.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35, pages 507–520.

Guo, C. (2015). Entity embedding: Rossmann competition code, Kaggle branch. GitHub repository, https://github.com/entron/entity-embedding-rossmann/tree/kaggle. Third-place solution; method in Guo and Berkhahn (2016), arXiv:1604.06737. README and `prepare_features.py` inspected 23 September 2026.

Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, third edition. Chapter 10, Dynamic regression models. https://otexts.com/fpp3/.

James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R*. Springer, 2nd edition.

Januschowski, T., Gasthaus, J., Wang, Y., Salinas, D., Flunkert, V., Bohlke-Schneider, M., and Callot, L. (2020). Criteria for classifying forecasting methods. *International Journal of Forecasting*, 36(1):167–177.

Kaggle (2015). Rossmann store sales forecasting competition. https://www.kaggle.com/c/rossmann-store-sales. Accessed June 2026.

Kang, Y., Hyndman, R. J., and Smith-Miles, K. (2017). Visualising forecasting algorithm performance using time series instance spaces. *International Journal of Forecasting*, 33(2):345–358.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.

Ma, S. and Fildes, R. (2022). The performance of the global bottom-up approach in the M5 accuracy competition: A robustness check. *International Journal of Forecasting*, 38(4):1492–1499.

Makridakis, S., Petropoulos, F., and Spiliotis, E. (2022a). The M5 competition: Conclusions. *International Journal of Forecasting*, 38(4):1576–1582.

Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2020). The M4 competition: 100000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74.

Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2022b). M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364.

Makridakis, S., Spiliotis, E., and Assimakopoulos, V. (2022c). The M5 competition: Background, organization, and implementation. *International Journal of Forecasting*, 38(4):1325–1336.

Makridakis, S., Spiliotis, E., Assimakopoulos, V., Chen, Z., Gaba, A., Tsetlin, I., and Winkler, R. L. (2022d). The M5 uncertainty competition: Results, findings and conclusions. *International Journal of Forecasting*, 38(4):1365–1385.

Montero-Manso, P. and Hyndman, R. J. (2021). Principles and algorithms for forecasting groups of time series: Locality and globality. *International Journal of Forecasting*, 37(4):1632–1653.

Ord, J. K. (2022). The uncertainty track: Machine learning, statistical modeling, synthesis. *International Journal of Forecasting*, 38(4):1526–1530.

Spiliotis, E., Kouloumos, A., Assimakopoulos, V., and Makridakis, S. (2020). Are forecasting competitions data representative of the reality? *International Journal of Forecasting*, 36(1):37–53.

Therneau, T. M. and Atkinson, E. J. (2023). An introduction to recursive partitioning using the RPART routines. Technical report, Mayo Foundation. R package `rpart` vignette.

Wright, M. N. and Ziegler, A. (2017). ranger: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1):1–17.